

온습도 변화를 고려한 저비용 교량 진동 이상 탐지 시스템

1 주제

1.1 개발 배경 및 필요성

노후 교량의 갑작스러운 붕괴 - 반복되는 사회 문제

2023년 4월, 준공 약 30년의 분당 정자교 보행로가 캔틸레버 부재의 부식·손상 누적으로 일상 보행 중 붕괴해 1명이 사망하고 1명이 다쳤다. 2026년 7월에는 포항 구룡포의 해상 인도교인 보릿돌교 약 50 m 구간이 바다로 무너졌는데, 최근 점검에서 C등급을 받고 지난해 보수·보강까지 마친 상태였으며 파도에 의한 피로 누적 등이 원인으로 조사되고 있다. 두 사고는 점검에서 징후가 관측되거나 보수가 이뤄지고도 다음 점검 전에 붕괴한 사례로, 정기 점검만으로는 구조물 상태 변화를 연속적으로 파악해 사고를 막기 어렵다는 점을 보여준다.



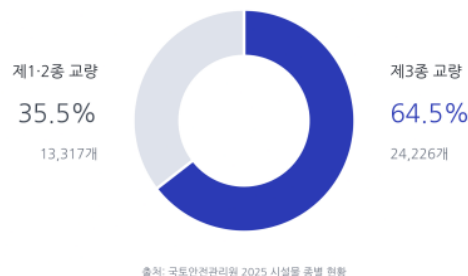
[자료1: 정자교 붕괴 현장 사진]



[자료2: 보릿돌교 붕괴 현장 사진]

두 사례처럼 노후 교량에서 장기간 진행되는 부식·균열·피로 누적은 강성·감쇠·고유진동수 같은 동특성 변화로 이어지므로 진동 기반 모니터링으로 관측할 수 있다. 그러나 진동 특성은 손상뿐 아니라 온도·습도 같은 환경 변수로도 변해, 관측된 변화가 손상 탓인지 환경 탓인지 구분하기 어렵다. 결국 점검과 점검 사이의 동특성 변화를 연속적으로 포착하는 일, 그리고 그 변화에서 환경의 영향을 분리해 손상에 의한 이상 징후만 가려내는 일이 과제로 남는다.

국토안전관리원 2025년 시설물 종별(규모에 따른 분류) 현황 기준 전국 교량 시설물은 제1종 5,392개, 제2종 7,925개, 제3종 24,226개로 총 37,543개이며, 상대적으로 규모가 작은 제3종 교량이 약 64.5%를 차지한다. 그만큼 안전관리 대상이 넓어, 중소형 교량에도 적용 가능한 저비용 상시 스크리닝 기술의 필요성이 커지고 있다.



[자료3: 전국 교량 중 제3종 교량의 비율]

중소형 교량의 상시 모니터링 사각지대

한편 교량의 상태는 규모와 별개로 안전등급(A~E)으로 평가되며, 법정 점검은 시설물의 현재 상태를 확인·평가하는 정밀안전점검과, 구조적 안전성을 공학적으로 검증하고 내하력(하중을 견디는 능력)까지 평가하는 더 깊은 단계인 정밀안전진단으로 나뉜다(자료4). 2025년 12월부터는 노후·저등급(C·D·E) 시설물까지 정밀안전진단 대상이

확대되었다. 그러나 정밀안전진단은 비용·인력 부담이 크고 안전등급에 따라 4~6년 등 일정 주기로만 시행되어, 점검과 점검 사이에는 시간적 공백이 남고 그사이 동특성 변화가 누적되어도 감지하기 어렵다. 결국 보행자 육교·소형 보도교 등 노후 중소형 교량에는 점검 사이 상태 변화를 상시 감지할 저비용 보조 모니터링 수단이 부족하다.

종류\안전등급	A등급	B·C등급	D·E등급
정기안전점검	반기 1회 이상	반기 1회 이상	1년 3회 이상
정밀안전진단	6년 1회 이상	5년 1회 이상	4년 1회 이상

[자료4: 시설물 등급별 점검 주기 및 대상 표]

정밀 진단 이전 단계의 저비용 스크리닝 기술 부재

기존 SHM(Structural Health Monitoring) 시스템은 센서를 구조물에 상시 설치해 연속 계측하는 기술이나, 초정밀 압전형 가속도계, 고성능 수집 장비, 광섬유 분포형 센서(KRIS BOCDA 등) 같은 정밀 계측 장비와 전용 인프라에 의존해 인천대교와 같이 규모와 중요도가 큰 일부 대형 교량 위주로 적용되며, 운영 중인 중소형 노후 교량에 저비용으로 보편 적용하기에는 경제성·시공성 제약이 크다. 그 결과 주기적 정밀 진단과 고비용 상시 계측 사이, 즉 정밀 진단 이전 단계의 저비용 상시 스크리닝 기술은 사실상 비어 있다. 따라서 교량 안전관리는 상시 스크리닝(1단계 선별)으로 이상 후보를 걸러낸 뒤 정밀안전점검·정밀안전진단으로 이어지는 다단계 개념으로 보완될 필요가 있으며, 그래야 한정된 점검 인력과 예산을 의심 교량에 집중할 수 있다.

1.2 개발의 주제 및 목적

본 작품은 앞서 정리한 세 문제, 즉 ① 점검 사이 동특성 변화의 연속적 포착과 환경 영향의 분리, ② 정밀 진단 주기 사이의 시간적 공백, ③ 정밀 계측 시스템의 중소형 노후 교량 적용 곤란의 해결을 목표로 한다. 붕괴를 예측하거나 정밀안전진단을 대체하는 시스템이 아니라, 정밀 점검의 우선순위를 제시해 점검과 진단 사이의 시간적 공백을 보완하는 저비용 1단계 스크리닝(보조) 시스템이다.

본 작품의 목적은 이 사각지대와 기술 부재를 동시에 해결하는 저비용 머신러닝 기반 교량 진동 스크리닝 시스템의 개발이며, 목표는 세 가지다. 첫째, ADXL355Z MEMS 가속도계와 Raspberry Pi(1차 3 Model A+, 2차 5)로 진동 데이터를 수집하되, 운영 중인 교량에서는 손상 데이터를 일부러 만들 수 없으므로 정상 데이터만으로 학습하는 비지도 학습을 택한다. 둘째, 교량 고유진동수는 손상이 없어도 기온·습도에 따라 오르내리므로[2], DHT22 및 DS18B20 센서로 기온·표면온도·상대습도를 함께 기록하고 정상 상태의 온습도-고유진동수 관계를 학습해 환경으로 설명되는 변동을 건어내 오탐을 줄인다. 셋째, 이상 신호만으로는 점검자가 어디를 살펴야 할지 알기 어려우므로, 어느 주파수 대역이 정상 기준을 벗어났는지 함께 제시해 후속 정밀 점검의 방향을 잡아 주는 설명 가능한 알람을 구현한다.

해결 방법

세 목표는 하나의 비지도 이상 탐지 파이프라인으로 구현한다. 1) 정상 상태의 진동 패턴만 학습해 두고, 2) 온습도로 설명되는 정상 변동을 먼저 건어낸 잔차가 학습된 패턴에서 크게 벗어나면 이상 후보로 판단하며, 3) 이때 어느 주파수 대역의 재구성 오차(모델이 학습한 정상 패턴으로 입력을 복원했을 때 원본과의 차이로, 정상에서 벗어날수록 커진다 — 원리는 2.3)가 큰지 시각화해 확정 진단 대신 점검 방향 판단을 돕는 해석 정보를 함께 제공한다. 하드웨어 구성은 2.2, 세 기능을 연결한 구체적 절차는 2.3 작동 원리에서 단계별로 다룬다.

1.3 유사 제품(연구) 및 차별점

저비용 MEMS 가속도 센서 기반 SHM 시스템

Raspberry Pi와 저가형 MEMS 가속도계(ADXL355Z)를 조합해 수천만 원대 초정밀 압전형 센서 없이도 구조물의 고유진동수·모드 형상·감쇠 특성을 식별할 수 있음을 입증한 SHM 연구가 있다.[1] 다만 이 연구는 동적 특성 식별에 초점을 두어, 온습도 등 환경 변수나 운영 하중 변화가 이상 탐지에 미치는 영향은 다루지 않는다.

광섬유 분포형 센서 시스템

한국표준과학연구원(KRISS)의 BOCDA 기반 광섬유 센서 시스템은 구조물의 변형률과 하중 위치를 높은 공간 해상도로 정밀 계측하는 기술로, [4] 전용 인프라를 전제로 한 정밀 변형률 계측이라는 점에서 측정 물리량과 적용 목적이 본 작품의 진동 기반 상시 스크리닝과 다르다.

비지도 학습 기반 SHM 알고리즘

스위스 Z24 Bridge Benchmark(KU Leuven 제공)는 장기 모니터링과 점진적 손상 시험 데이터를 포함한 대표적 공개 데이터셋이다. [3] Autoencoder, One-Class SVM, Isolation Forest, VAE 등 다양한 비지도 학습 모델이 Z24에서 비교되어 왔으나 [5][6], 대부분 이상 점수나 임계값 초과 여부 출력에 그쳐 어느 대역이 기준을 벗어났는지는 설명하지 않는다.

실규모 교량 장기 모니터링 데이터 기반 환경 보정 연구

독일 바우첸(Bautzen)의 실규모 연구용 교량을 장기간 모니터링한 openLAB Research Bridge 공개 데이터셋은 가속도 응답과 기온·습도 등 환경 측정값을 함께 제공해 계절 변화·일교차의 영향 분석에 적합하다(상세는 2.7 참조).[8] 본 작품에서 openLAB은 환경 보정 원리가 실규모 장기 데이터에서도 타당한지 확인하는 보조 검증 데이터이며, 기여의 중심은 수집·보정·판단·설명을 한 파이프라인으로 연결한 저비용 하드웨어 구현과 자체 모형 교량 실측 검증에 있어 공개 데이터 재분석 연구와 구별된다.

따라서 이 작품의 차별성은 저비용 하드웨어 직접 구현, 온습도 환경 보정, 대역별 설명 가능성의 세 축으로 정리된다. 즉 본 작품은 저비용 MEMS 연구와 달리 온습도 보정으로 오차를 줄이고, 광섬유 계측과 달리 저가 범용 부품으로 상시 스크리닝을 수행하며, 이상 점수만 출력하는 비지도 학습 연구와 달리 대역별 재구성 오차 기여도까지 제시한다.

2 작품 제작

2.1 목표

이 시스템은 저비용 MEMS 가속도계(ADXL355Z)와 온습도 센서(DHT22)로 진동·온도·상대습도를 함께 수집하고, 정상 상태에서 학습한 온습도·진동수 관계로 환경에 따른 오차를 줄인 뒤, Autoencoder로 정상 패턴에서 벗어난 이상 후보를 탐지해 재구성 오차가 큰 주파수 대역과 함께 제시하는 보조용 진동 스크리닝 시스템이다.

자체 실험은 세 단계로 진행한다. 1차는 폼보드 모형과 수동 가진으로 수집·보정·탐지·설명 파이프라인 전체의 동작을 확인한다. 2차는 1차의 두 한계(공진 피크 불선명, 수동 가진 편차)를 알루미늄 다경간 모형과 솔레노이드 오토 태퍼로 개선해 온도·습도·질량·볼트 풀림 조건 매트릭스에서 구분 성능을 검증하고, Raspberry Pi 5 기반 실시간 측정·분석 체계를 함께 구축한다. 3차는 이 시스템을 실제 육교 2곳에 옮겨 환경 영향과 검출력을 확인한다(상세는 2.2·3.1 참조).

2.2 작품 설계

하드웨어 구성

본 작품은 하드웨어 수집부와 소프트웨어 분석부로 나누어 제작하며, 하드웨어 구성 요소와 설계 이유는 다음 표와 같다.

구성 요소	역할·설계 이유
ADXL355Z 3축 가속도계	3축 진동 측정 — 고유진동수 등 진동 특성 변화 감지
DHT22 온습도 센서[7]	온습도 측정 — 환경 기인 정상 변동 보정
Raspberry Pi 3 Model A+ (1차)	데이터 수집·저장 (Python 로깅)
Raspberry Pi 5 (2차)	실시간 수집·태퍼 제어·FFT·이상 점수·대시보드를 한 장치에서 실행
폼보드 교량 모형 (1차)	질량 분포 등 구조 조건 실험용 1차 모형
알루미늄 교량 모형 (2차)	감쇠가 작아 공진 피크 선명, 볼트 체결로 경계조건 반복성 확보
솔레노이드 오토 태퍼 (2차)	일정 세기·주기·위치 자동 가진 — 수동 편차 제거로 정상 산포 축소
DS18B20 표면온도 센서 (3차)	데크 표면 온도 측정 — 환경 보정 입력 후보로 기록 (방수 프로브, 4,200원)
LTE 라우터 (3차)	현장 노드 원격 접속·데이터 확인 (에듀플레이어 ELR22, 59,650원)
보조배터리 (3차)	현장 무전원 설치용 전원 공급 (30,000 mAh, 20,900원)

핵심 계측·처리 장치의 비용은 ADXL355Z 평가보드 84,589원, DHT22 온습도 센서 5,680원, Raspberry Pi 5(RAM 2~4 GB 모델 기준 약 100,000원)로 합계 약 19만 원이며, 3차에 추가한 DS18B20 표면온도 센서(4,200원)와 현장 운용용 LTE 라우터(59,650원)·보조배터리(20,900원)를 더해도 약 27만 5천 원이다. 이는 초정밀 압전형 가속도계와 전용 수집 장비를 전제로 하는 수천만 원대 기존 SHM 계측(1.3·[1] 참조)보다 크게 낮아, 중소형 교량 다수에 보급 가능한 저비용 구성이라는 본 작품의 목표를 뒷받침한다.

소프트웨어 구성

소프트웨어 분석부는 수집·분석 두 프로그램으로 구성한다. 수집 프로그램은 Raspberry Pi에서 Python으로 동작하며 ADXL355Z 3축 가속도를 200 Hz(2·3차 실험은 500 Hz)로, DHT22 온도·상대습도를 약 2초 간격으로 한 CSV 파일에 기록한다. 분석 프로그램은 이 파일을 시간 창(window) 단위로 나눠 FFT·피쳐 추출, 온도 T·상대습도 RH 다변량 회귀 잔차 계산, 표준화, 재구성 오차 기반 이상 점수 산출, 99번째 백분위수 임계값 판정, 대역별 기여도 출력 순으로 동작한다.

2차 실험에서는 이 구성을 Raspberry Pi 5 기반의 실시간 시스템으로 확장한다. 수집 로거·태퍼 제어·분석 서버를 같은 장치에서 실행해 동일 시계를 보장하고, 타격 직후 창(타격 시각 t_0 기준 $t_0+0.05$ s부터 2.0 s 동안)을 잘라 스펙트럼과 이상 점수를 즉시 계산해 웹 대시보드로 표시한다. 대시보드는 실시간 z축 파형·타격 스펙트럼·임계값 대비 배율(이상 점수 ÷ 임계값)·피크 주파수·RMS·온습도·대역별 기여도를 약 0.7초 주기로 갱신하며, 배율 0.8 미만은 정상, 0.8~1.0은 주의, 1.0 이상은 이상 후보 상태로 표시한다. 실시간 판정은 현장 모니터링용이고, 원본은 조건 코드를 포함한 CSV로 전부 저장해 모델 학습·평가에 사용한다.

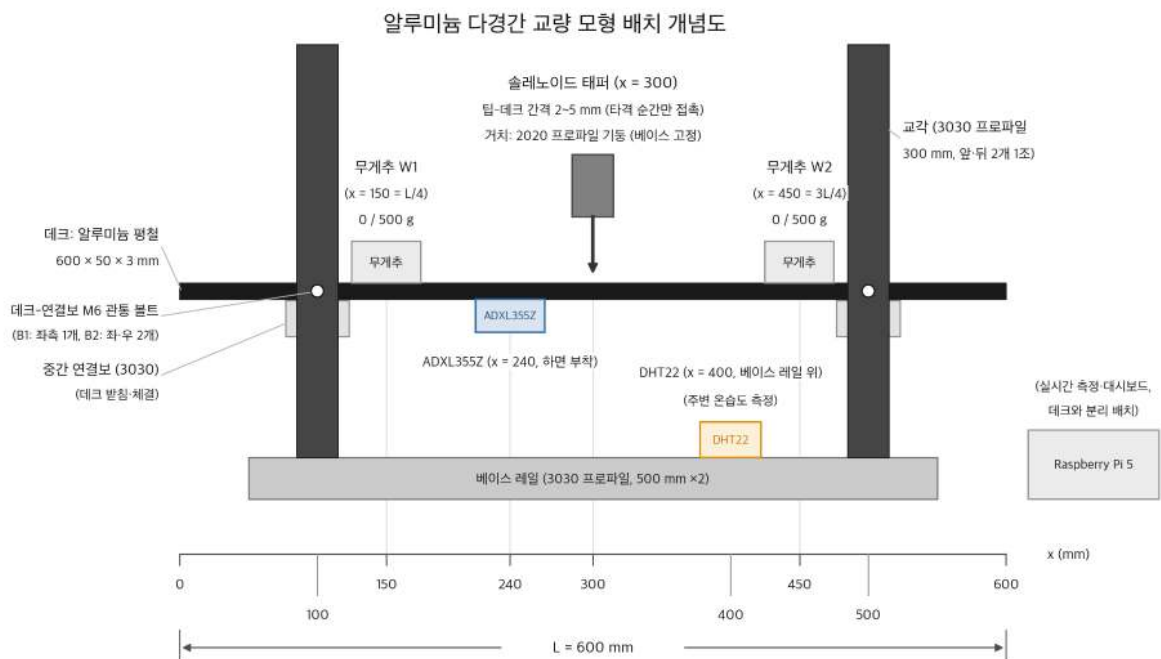
3차 실험교량에서는 태퍼 없이 상시 진동을 연속 기록하도록 수집 프로그램을 바꿔 DS18B20 표면온도를 추가하고, 장시간 무인 수집에 맞춰 CSV를 1시간 단위로 나눠 저장한다. 이상 탐지는 현장에서 실시간으로 하지 않고 수집 후 일괄 수행한다.

센서 부착 위치

1차 폼보드 모형에서 ADXL355Z는 1차 휨 응답이 최대인 중앙부(경간 L·폭 B 기준 $x = L/2$, $y = B/2$)에 부착하고, 상하 휨을 반영하는 수직 z축을 주요 해석 대상으로 삼는다.

2차 알루미늄 모형에서는 데크 좌측 끝단을 $x = 0$ 으로 하는 길이 방향 좌표(데크 전장 600 mm)로 모든 위치를 정의한다. 슬레노이드 태퍼의 타격점은 중앙 $x = 300$ mm 상면 폭 방향 중앙으로 하여 매 타격이 지배 모드를 일관되게 가진하도록 하고, 팁과 데크 사이 간격은 카드 게이지로 2~5 mm를 전 조건 동일하게 유지해 타격 순간에만 접촉하도록 한다. ADXL355Z는 타격점 직하부를 피한 $x = 240$ mm의 데크 하면에 부착해, 타격 직후의 국부 충격 성분과 상면 무게추·타격점과의 간섭을 피하면서 중앙 인근의 큰 휨 응답을 측정한다. 무게추는 1차 실험과 같은 비율인 $x = 150$ mm($L/4$)와 $x = 450$ mm($3L/4$) 두 지점에 위치시킨다(그림 1).

DHT22와 Raspberry Pi는 두 실험 모두 교량의 질량·강성에 영향을 주지 않도록 진동 구조(데크·교각) 밖에 둔다. 1차에서는 모형 옆에 두고 케이블을 여유 있게 고정했으며, 2차에서는 DHT22를 베이스 레일 위 $x = 400$ mm 지점에 고정해 교량 주변의 온도·상대습도를 전 조건 동일한 위치에서 기록하고, Raspberry Pi는 데크에서 떨어진 베이스 옆에 두어 배선 장력과 질량이 진동 응답에 영향을 주지 않도록 한다.



[그림 1: 알루미늄 교량 모형 배치 개념도(2차 실험 기준)]

2.3 작동 원리

1. 진동 및 온습도 데이터 수집: ADXL355Z에서 3축 가속도를, DHT22에서 온도 T와 상대습도 RH를 수집한다. 예비 실험으로 실제 고유진동수 범위를 확인해 분석 대역을 나이퀴스트 한계 이내에서 설정하며, 실측 기본 고유진동수가 200 Hz의 나이퀴스트 한계(100 Hz)에 근접·초과할 경우 샘플링 주파수를 500 Hz로 상향하도록 계획하였다. 2차 알루미늄 모형은 조립 후 예비 측정에서 지배 모드 주파수가 이 한계를 넘어, 샘플링 주파수를 500 Hz로, 측정 레인지를 ± 8 g로 상향 확정하였다(구체적 실측 고유진동수는 3.4에 기입). 가진은 1차 수동 타격, 2차

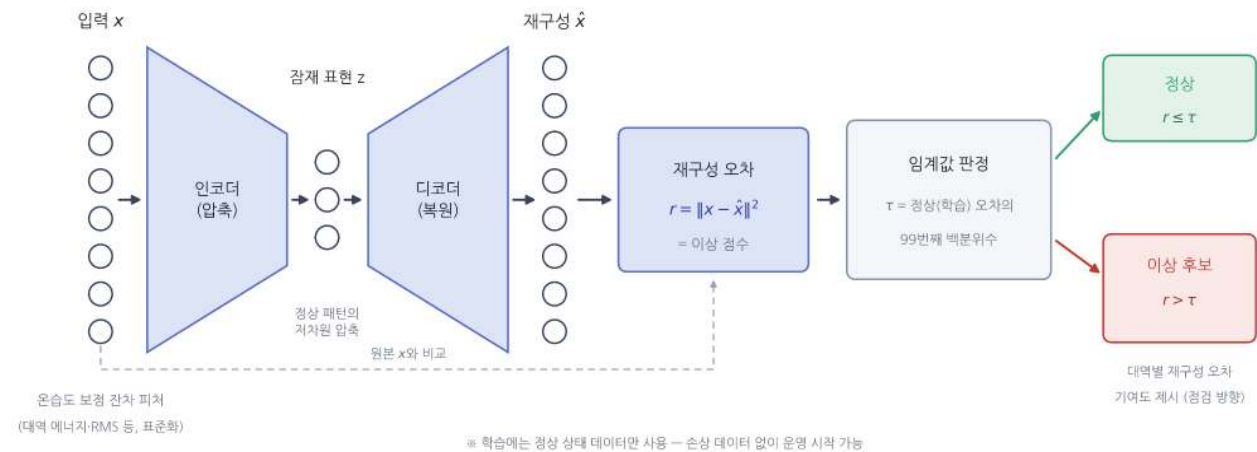
슬레노이드 오토 태퍼의 자동 타격(워밍업 후 일정 주기)으로 수행하고, 타격 시각을 동일 시계로 기록해 타격 직후의 자유진동 구간만 분석 단위로 사용한다.

2. 신호 전처리와 주파수 대역 피쳐 추출: 가속도 신호에서 DC offset을 제거하고 스펙트럼을 추정한다(1차 FFT/STFT, 2차는 신호를 짧은 구간으로 나눠 스펙트럼을 평균함으로써 잡음 변동을 줄이는 Welch 방법).

구간·축별로 RMS, 표준편차, peak-to-peak, 피크 주파수·파워, 대역별 에너지 등을 계산하며, 분석 대역은 실측 지배 피크에 맞춰 재설정하며, 2차 이후에는 진폭·에너지를 로그로, 대역 에너지를 비율로 정규화한 z축 8개 피쳐를 쓴다(대역과 피쳐 목록은 3.2). 온도 T·상대습도 RH는 환경 보정에 사용한다. z축 피크 주파수를 지배 모드 고유진동수의 지표로 삼으며, 대역별 분석은 손상 위치 특성이 아니라 어떤 진동 특성이 기준을 벗어났는지 설명하는 보조 지표다.

3. 온도·습도 기반 정상 고유진동수 다변량 회귀 보정: 정상 데이터에서 온도 T와 상대습도 RH를 설명 변수로 고유진동수를 예측하는 다변량 회귀를 학습하고, 추론 시 예측값 $\hat{f} = g(T, RH)$ 와 실측값 f 의 차이 $r = f - \hat{f}$ 를 온습도 효과를 보정한 잔차로 정의한다. 같은 회귀·잔차 계산을 각 진동 피쳐에 적용해 얻은 잔차 피쳐들을 이상 탐지 모델의 입력으로 쓴다. 잔차에는 구조적 이상 후보 외에 센서 노이즈·부착 상태·운영 하중의 영향도 남을 수 있어, 손상 확정값이 아니라 이상 후보 탐지 기준으로 사용한다.

4. Autoencoder 기반 비지도 학습 이상 탐지: 표준화한 정상 상태의 잔차 피쳐만으로 Autoencoder를 학습한다(구조는 그림2 참조). 모델은 입력을 저차원 잠재 표현으로 압축·복원하며 정상 데이터의 상관 구조만 학습하므로, 정상 패턴에서 벗어난 데이터는 재구성 오차가 커진다. 임계값은 정상 검증 데이터 재구성 오차의 99번째 백분위수로 설정하며(2차에서는 학습에 쓰지 않은 임계값 전용 정상 창 90개의 오차 분포 사용, 3.2 참조), 학습과 임계값 설정이 모두 정상 데이터만으로 이루어져 손상 데이터 없이 운영을 시작할 수 있다. 1차 구현은 선형 Autoencoder와 동일한 PCA 재구성 오차를 사용하였고, 2차에서는 은닉 8-3-8 구조의 비선형 Autoencoder(tanh, 조기 종료)를 채택하였다.



[그림2: Autoencoder 기반 이상 탐지 구조]

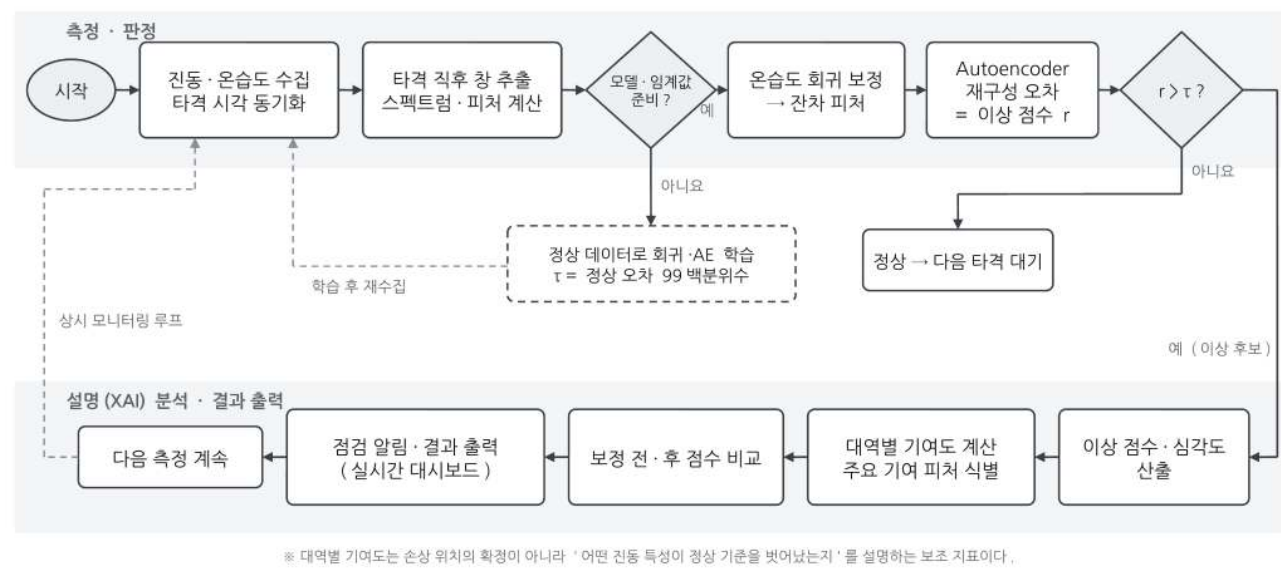
5. 대역별 재구성 오차 기여도 분석: 이상 감지 시 어느 주파수 대역(또는 모드 주변 대역)의 재구성 오차가 큰지 함께 분석한다. 기여도는 “해당 피쳐의 재구성 오차 ÷ 전체 재구성 오차”로 계산한다.

6. 결과 출력 및 정밀 점검 알림: 이상 점수가 임계값을 초과하면 알람과 함께 이상 점수, 온습도 보정 전후 비교, 주요 기여 대역, 고유진동수 변화율, 대역별 기여도, 점검 참고 해석을 출력한다. 초과 여부(탐지)와 별도로 임계값 대비 비율(2.2)과 최근 구간의 임계값 초과 비율을 심각도 지표로 산출해 정밀 점검 우선순위 근거로 제시하되, 이

지표는 손상의 물리적 심각도가 아니라 정상 상태 대비 이탈 정도를 나타내므로 서로 다른 손상 유형 간 비교에는 사용하지 않는다(검증 계획은 3.3 참조).

2.4 시스템 순서도

위 모듈을 소프트웨어 단위로 연결한 작동 구조는 아래 시스템 순서도(그림3)와 같다. 수집 → 전처리·피쳐 추출 → 환경 보정 → 이상 점수 산출까지는 매 분석 창마다 반복되는 상시 경로이고, 이상 점수가 임계값을 초과하는 경우에만 대역별 기여도 분석과 정밀 점검 알림 경로로 분기한다. 각 블록은 2.3의 1~6단계에 순서대로 대응한다.



[그림3: 교량 진동 이상 탐지 시스템 순서도]

2.5 제작 과정

단계	내용	산출물
1단계 (06/03)	센서(ADXL355Z·DHT22·Pi 3A+) 연결 확인, 폼보드 모형 제작·센서 위치 결정	연결 테스트 결과·부착 사진
2단계 (06/03~06/04)	정상(0-0)·질량 조건(1-0, 1-1) 데이터 수집(표기는 3.1 참조)	조건별 raw CSV
3단계 (06/04)	환경 조건 수집(야외 온도 제외)·9개 조건 정리	9개 조건 데이터셋(기준·습도·온습도 동시)
4단계 (06/04~06/05)	FFT/STFT 피쳐 추출, Autoencoder 학습 및 이상 점수 계산	피쳐 데이터셋·탐지 결과
5단계 (06/05~06/12)	대역 기여도 분석·1차 결과 정리	기여도 그래프·1차 보고서
6단계 (06/17~07/08)	2차 부품(알루미늄盒 모형·태퍼) 구매·수급	부품 일체
7단계 (07/11~07/12)	모형 조립(레일-교각-연결보-데크), 태퍼 회로·Pi 5 실시간 대시보드 구현	2차 모형·과정 사진
8단계 (07/13~07/18)	정상 기준 데이터 수집 및 27조건 매트릭스 실험	조건별 CSV·타격 기록
9단계 (07/19~07/26)	2차 결과 분석, 모델 비교 및 설명서 반영	결과 표·그래프·최종 설명서
10단계 (09/07~09/25)	실교량 E·C 3차 실험, 추가질량(100·200 kg) 실험	세션별 CSV·분석 결과

2.6 제작 항목 및 핵심 구현

하드웨어 제작

1차 실험에서는 가로 약 55 cm 폼보드 모형에 ADXL355Z·DHT22·Raspberry Pi 수집 체계를 구성해 온습도 보정 이상 탐지의 개념을 검증하였다. 다만 폼보드는 감쇠·조립 편차로 공진 피크가 완만해, 2차에서는 같은 센싱·보정 체계를 알루미늄 다경간 모형으로 확장하였다.



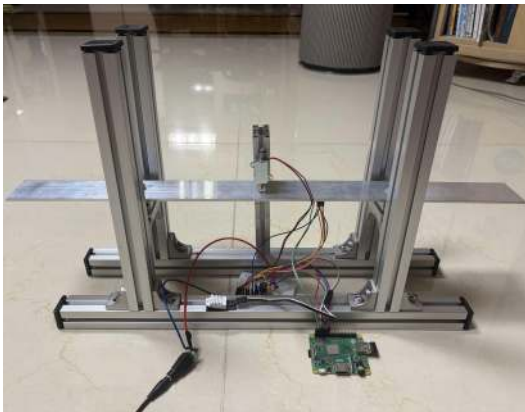
1차 실험용 폼보드 교량 모형 (가로 약 55 cm)



ADXL355Z 3축 MEMS
가속도계



DHT22 온·습도 센서



2차 실험용 알루미늄 교량 모형(가로 약 60cm)



솔레노이드 오토 태퍼



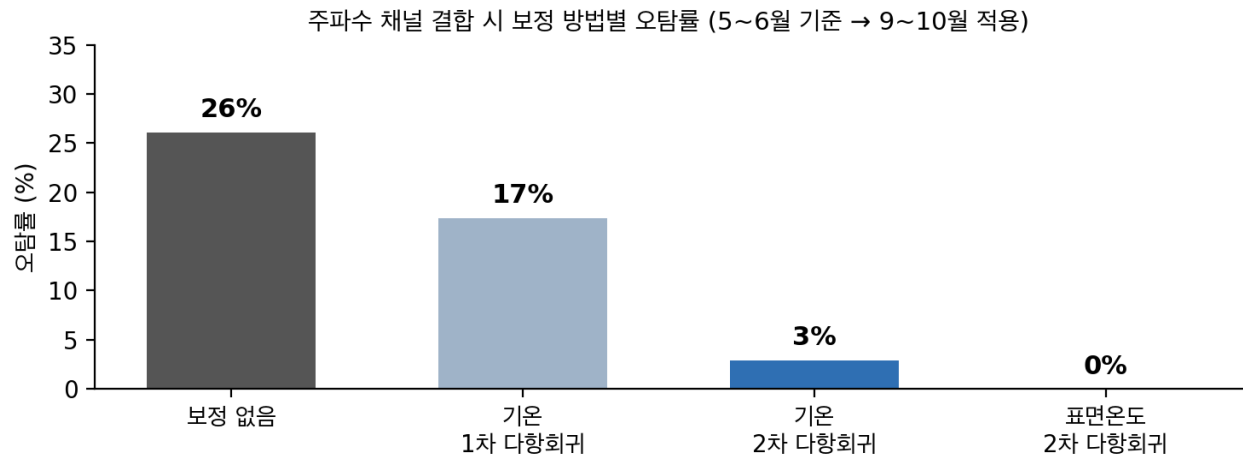
Raspberry Pi 5 실시간 대시보드

2.7 openLAB 실규모 데이터 검증

오토인코더에 주파수 판정 채널을 결합했을 때 계절 변화에서 온도 보정이 어떻게 작동하는지는 TU Dresden이 공개한 openLAB 데이터셋[8]으로 확인하였다. 대상은 약 45 m의 3경간 콘크리트 연구용 교량으로, 2024년 봄~가을의 기온·습도와 가속도 기록 526회가 제공되며 전 기간 손상이 없으므로 경보는 모두 오탐이다. 각

기록에서 구한 고유진동수 세 개는 모두 기온이 내려가면 높아졌고(7.05 Hz 모드 약 $-0.19\%/^{\circ}\text{C}$), 15°C 아래에서 상승이 가팔라져 10월에는 6~8%까지 올랐다. 여름(5~6월) 기록으로 정상 기준과 임계값을 정해 가을(9월 중순~10월) 기록 69개에 그대로 적용해 보정 방법별 오تام률을 비교하였다. 판정은 현재의 8피치 오토인코더에, 온도 영향이 가장 뚜렷한 7.05 Hz 모드의 보정 주파수가 여름 기록의 중앙 99% 범위(양쪽 끝 0.5%씩 제외)를 벗어나는지 보는 주파수 채널을 결합해, 둘 중 하나라도 넘으면 경보로 세었다. 두 채널 모두 정상 기록의 99%를 기준으로 삼는다.

보정 방법 (결합 판정)	오تام률: 학습 구간 (5~6월, 184개)	오تام률: 평가 구간 (9~10월, 69개)	평가 구간 중 학습 최저 기온보다 추운 기록 (40개)
보정 없음	1.6%	26.1%	42.5%
기온 1차 다항회귀	1.6%	17.4%	27.5%
기온 2차 다항회귀	1.6%	2.9%	2.5%
표면온도 2차 다항회귀	1.6%	0.0%	0.0%



[그림4: 주파수 채널 결합 시 openLAB 9~10월 69개의 보정 방법별 오تام률 (여름 학습 기준 고정 적용)]
기존 오토인코더 단독으로는 10월에 주파수가 6~8%까지 올랐는데도 이상 점수가 임계값의 약 0.1배에 머물러 계절 변화 자체를 보지 못했다. 주파수 채널을 결합하고 보정하지 않으면 가을 기록의 26%가 이상으로 판정되어, 선행연구[2]에서 보고된 미보정 오تام 급증이 재현되었다(그림4). 기온 1차 다항회귀 보정은 오تام률을 17%로 낮추는 데 그쳤는데, 여름 구간에서 학습한 선형 관계로는 저온 영역의 비선형 상승을 외삽할 수 없기 때문이다. 2차 다항회귀 보정에서는 오تام률이 3%로, 표면온도 기반 2차 보정에서는 0%로 감소해 학습 구간(약 2%) 수준으로 돌아왔다. 상대습도는 기온 2차 모델에 추가해도 오تام률이 그대로여서(3→3%) 제외하였으며, 이는 고유진동수 변동이 주로 온도로 설명된다는 선행연구[2]의 결과와 부합한다.

3 작품 테스트

3.1 테스트 목적 및 실험 설계

본 테스트의 목적은 세 가지이다. 첫째, 저비용 센서(2.2)와 모형 교량으로 구조 조건 변화가 진동 피치·이상 점수에 주는 영향을 확인한다. 둘째, 온습도 보정의 오تام 감소를 검증한다. 실규모 비교는 openLAB(2.7)이 맡는다. 셋째, 검증한 시스템을 실제 육교 2곳에 옮겨 교량 E에서는 환경 영향을, 교량 C에서는 검출력을 확인한다(3차). 성능 입증의 중심은 조건을 통제할 수 있는 2차이며, 3차는 손상 라벨이 없는 현장의 보조 검증이다.

1차 실험 설계 — 품보드 모형과 질량 조건

L/4·3L/4 지점에 100 g 무게추를 두어 0-0(정상)·1-0(좌측)·1-1(양측) 세 조건을 만든다.

2차 실험 설계 — 알루미늄 다경간 모형과 27조건 매트릭스

아래 요인을 환경 3상태 × 질량 3수준 × 볼트 3수준 = 27조건 매트릭스로 수행한다. 환경은 전열기가 온도·습도를 동시에 바꿔 온도 단독 변화를 부여할 수 없다. 이에 변화없음(Tx-Hx)·습도변화(Tx-Ho)·온습도변화(To-Ho)의 세 상태로 구성한다. 이 설계는 환경(T·H)과 손상 모사(M·B)의 교호작용 데이터로, 환경 변화는 정상으로·손상은 이상으로 구분하는 온습도 보정의 가치를 검증한다.

요인(코드)	수준	부여 방법
환경 상태 (T·H)	변화없음 / 습도변화 / 온습도변화 (3수준)	Tx-Hx = 환경 부여 없음. Tx-Ho = 불투명 박스+가습기로 고습 부여(온도 유지). To-Ho = 전열기 가열로 온도 상승·습도 하강. 분석에는 DHT22 실측값 사용
질량 변화 (M)	0 / 500 / 1,000 g (3수준)	x = 150·450 mm 두 표식에 M500은 한쪽 500 g 비대칭, M1000은 양쪽 각 500 g 대칭 부여. 50 g 무게추를 매번 같은 순서로 적재
볼트 풀림 (B)	0 / 1 / 2개 (3수준)	데크-연결보 관통 볼트를 완전 조임에서 1/2바퀴씩 풀기. B1 = 좌측 1개(비대칭), B2 = 좌·우 2개(대칭 경계 손상 모사)

풀림은 데크 경계조건에 직접 작용하는 데크-연결보 관통 볼트로 한정하고, B1이 B2의 부분집합이 되도록 구성해 경계 강성을 단계적으로 낮춘다. 볼트 조건 종료 후 원래 조임으로 복원해 이상 점수가 기준선으로 돌아오는지 확인한다. 정상 클래스는 M0·B0인 3셀(환경 3상태)로 정의해 환경만 변한 상태를 정상으로 학습시키며, 학습·임계값 산정용이라 다른 조건보다 길게 수집한다.

실행은 환경 3블록으로 묶어 블록별 질량×볼트 9조건을 수행한다. 조건당 위밍업 10회(미수집) 뒤 본 타격 35회를 가하고, 정상 3셀은 본 타격 180회씩 수집한다. 정상 셀은 시간 순서로 학습 120·임계값 30·평가 30창으로 나뉘, 합계 학습 360·임계값 90·평가 90창을 확보한다.

3차 실험 설계 — 실교량 방문형 실측

후보 육교 5곳(A~E)에서 상시 진동을 30분씩 측정해 아래 표와 같이 비교했다. 지배 피크가 대역 중앙값 대비 1만 배 이상 선명한 E(1.74 Hz)와 C(3.81 Hz)를 대상으로 골랐고, 피크가 보이지 않는 D, 진동이 너무 작은 A, 충격성 진동이 섞인 B는 제외했다. E는 며칠 관측이 가능해 환경 검증에, C는 재하가 가능해 검출력 검증에 배정했다.

교량	지배 피크	피크 선명도 (대역 중앙값 대비)	z축 변동 (mg, 1σ)	판정
E	1.74 Hz	17,369배	1.36	선정 (1순위)
C	3.81 Hz	17,632배	2.57	선정 (2순위)
B	3.69 Hz	1,698배	7.27	제외 (충격성 진동 혼재)
A	2.62 Hz	194배	0.43	제외 (진동 과소)
D	피크 없음	3.3배	0.81	제외 (피크 식별 불가)

가진은 보행·차량이 만드는 상시 진동이며, 단일 노드에 데크 표면온도 센서(DS18B20)를 추가했다. 교량 E는 환경 검증을 맡아 9월 7~10일의 4일 동안 7세션, 약 68시간을 수집했다. 실교량에는 손상을 가할 수 없어 재현율은 산출하지 않고, 최근 점검 등급을 근거로 정상 상태를 가정해 임계값 초과를 오염으로 간주한다. 주요 지표는 모드

피크의 재현성, 한 세션의 임계값을 다른 세션에 고정 적용한 오탐률, 피크 주파수의 온도 민감도다. 교량 C는 검출력 검증을 맡아 가역적 추가질량 실험을 수행했으며, 9월 20일 같은 부착 3세션(약 6시간)의 예비 측정으로 추적할 피크를 확인했다. 필요한 무게는 주파수가 $\sqrt{(\text{강성}/\text{질량})}$ 에 비례하는 관계에서 정한다. 재하 위치에서 해당 모드에 유효하게 작용하는 질량을 등가질량 M_{eq} 라 하면, 이보다 충분히 작은 무게 m 은 주파수를 약 $m/2M_{eq}$ 의 비율만큼 낮추므로, 측정 오차보다 충분히 큰 목표 감소율 δ 에 필요한 무게는 $m \approx 2\delta M_{eq}$ 이다. M_{eq} 는 도면 없이는 알 수 없어, 현장에서 허용된 작은 무게 m_p 로 예비 재하를 해 무재하·재하 주파수 $f_0 \cdot f_p$ 에서 $M_{eq} \approx \frac{m_p}{(f_0/f_p)^2 - 1}$ 로 역산한 뒤 본시험 질량을 정한다.

3차의 수집 프로그램과 기록 주기는 2.2와 같되 조건 코드는 쓰지 않는다. 세션별 원시 CSV는 타임스탬프, DHT22 기온·상대습도, DS18B20 표면온도, ADXL355Z 내부 온도, g 단위 3축 가속도를 열로 갖고 폴더명의 세션 시작 시각으로 구분하며, 부착·재부착과 통행 사건은 현장 기록으로 남겨 분석 시 창과 연결한다.

3.2 테스트 방법

테스트는 openLAB 공개 데이터 분석(2.7)과 자체 모형 교량 실험, 실교량 실측으로 나눈다. openLAB은 전 구간이 무손상이어서 보정 방식별 오탐률 비교에 쓰고, 자체 실험은 저비용 센서 수집·질량 변화 탐지·보정 재현을 검증한다. 아래는 실험별 차이만 기술한다.

1차는 0-0·1-0·1-1을 수동 가진해 조건당 20~30초 이상 반복 기록하였다(환경 3조건, 4초 창). 모델은 정상 0-0 데이터만으로 학습하고, 임계값은 정상 재구성 오차의 99번째 백분위수를 세 모델(무보정·온도·온습도 보정)에 동일 적용한다. 세 모델은 2.3의 PCA 재구성(선형 AE 등가)이다.

2차의 핵심 변경은 수동 가진을 솔레노이드 오토 태퍼 자동 타격으로 바꿔 정상 기준의 산포를 줄인 것이다. 타격 시각과 가속도를 동일 시계로 동기화해 타격 직후 창만 분석한다.

샘플링 500 Hz·측정 레인지 ± 8 g로 수집하고, 분석 창은 임팩트를 제외한 $t_0+0.05$ s부터 2.0 s(1,000샘플)로 고정한다. 스펙트럼은 Welch로 추정하고 분석 대역은 실측 지배 모드(101 Hz)에 맞춰 0~40·40~80·80~125 Hz로 재설정했다. 피쳐는 z축 8개(로그 RMS·로그 p2p·피크 주파수·로그 피크 파워·로그 총에너지·대역 에너지 비율 3종)로, 진폭은 로그, 대역 에너지는 비율로 정규화해 타격 세기가 아니라 스펙트럼 구조 변화가 드러나게 했다.

학습·임계값·평가는 데이터 누수를 막도록 분리한다. 학습은 정상 360창, 임계값은 학습에 쓰지 않은 전용 90창의 재구성 오차 99번째 백분위수, 평가는 정상 90창·손상 840창으로 수행한다. 모델은 은닉 8-3-8 비선형 Autoencoder(tanh)로 통일하고 환경 입력만 무보정·온도·온습도로 바꿔 비교하며, 난수 시드 5개로 반복해 안정성을 확인한다.

3차는 500 Hz· ± 2 g로 수집한 z축 신호를 20 Hz 저역통과 후 50 Hz로 다운샘플링하고, 2차와 같은 8개 피쳐를 분석 대역만 각 교량의 지배 피크에 맞춰 20 Hz 이하로 재설정해 산출하며, 두 교량 모두 120초 창을 10초 간격으로 이동한다. 모형 실험에 쓰던 4초 창은 짧은 타격 기록에 맞춘 길이라 수십 분씩 이어지는 실교량 상태 변화에는 짧다. 교량 E는 한 세션에서 학습하고 임계값을 정한 뒤 같은 부착의 다른 세션에 고정 적용하며(재부착 세션 사이도 확인), 무보정·기온·기온+습도·표면온도 추가의 네 모델을 비교한다. 판정은 창 하나가 아니라 연속 창의 초과로 하여 한순간 여러 사람이 지나가는 단일 사건의 오탐을 막는다. 교량 C의 경보 모델은 재하 시험과 가진 조건을 맞춰 9월 25일 같은 위치의 무재하 걷기 30분 중 앞 20분으로 학습하고 뒤 10분으로 오탐률을 확인한다. 검출력 실험은 예비 재하 100 kg으로 M_{eq} 를 역산해 본시험 질량을 200 kg으로 정하고, 무재하 걷기 → 재하 걷기 30분 → 재하 힐드롭(발뒤꿈치 낙하 가진) → 제거 후 힐드롭 순으로 진행했다(야간이라 통행이 적어 실험자

보행으로 가진). 정보는 무재하 걷기로 학습한 모델을 그대로 적용해 판정하고, 원인은 힐드롭 뒤 저절로 잦아드는 진동(자유감쇠)의 주파수 하락·복귀로 판정한다.

3.3 예상 결과

1차에서는 질량 증가로 피크 주파수가 낮아지고, 비대칭 조건(1-0)에서는 진폭·대역 에너지 변동이 먼저 나타날 것으로 예상된다. 2차에서는 무게추가 지지점 부근에 놓여 유효 질량이 작으므로 지배 모드 주파수가 뚜렷이 하락하되 질량 비율만큼 떨어지지는 않고, 볼트 풀림은 고유진동수의 소폭 하락과 함께 고조파·피크 폭 확대를 동반할 것으로 본다. 특히 같은 손상 유형 안에서 조건이 심해질수록(M0 → M500 → M1000, B0 → B1 → B2) 이상 점수가 단조 증가하는지를 중점적으로 확인하며, 이 서열이 성립해야 이상 점수를 심각도 지표로 쓸 수 있다. 3차에서는 선행연구[2]와 같이 무보정 모델의 오탐률이 온도가 다른 세션에서 커지고 보정 모델이 이를 줄일 것으로 예상하되, 초가을 관측은 15 °C 아래 저온 구간을 포함하지 않아 그 효과가 작을 수 있다. 추가질량 실험에서는 고유진동수가 질량에 따라 하락했다가 제거 후 복귀하고, 약 0.4% 이상의 변화는 구분될 것으로 예상된다.

3.4 실제 결과

1차 폼보드 모형은 예비 측정의 기본 모드 피크(약 22.6 Hz)에 맞춰 분석 대역을 0~12·12~25·25~50 Hz로 설정하였다. 양측 하중(1-1)에서는 피크 주파수가 8.9% 낮아졌고, 한쪽 하중(1-0)은 피크 변화가 편차 범위 안이었지만 다피쳐 이상 점수로 감지되어 임계값 초과 비율이 29.7%까지 올랐다. 습도 변화 구간의 오탐률은 온습도 보정으로 17.5%에서 9.1%로 줄었고, 전체 평가 구간에서도 보정 모델이 무보정보다 낮았다(9.2% → 온도 4.8%·온습도 6.3%). 다만 온습도 보정 모델의 재현율은 0.234에 그쳐 손상 창 대부분을 놓쳤다.

2차 실험 결과

온습도 보정 모델은 손상 840창을 모두 검출하고(재현율 100%) 정상 평가 90창 중 4창(4.4%)에서만 오탐을 냈으며(ROC-AUC 0.996), 4건 모두 온습도 동시(To·Ho) 블록에서 나왔다(아래 표의 4/30). 정상 기준의 지배 모드는 101.0 Hz이고, 표의 손상 조건군은 환경 3블록 합산(각 105창)이다. 한편 피크 이동률이 3.3의 예상보다 훨씬 크고(-64.4% 등) 역전도 있는 것은(M1000·B1 -19.8%) 실제 고유진동수 하락이 아니라, 가장 큰 봉우리 하나만 읽는 피크 주파수가 손상으로 모드별 세기가 바뀌며 101 Hz 모드에서 다른 모드로 옮겨 앉았기 때문이다(모드 전환, 3.5). 정상 가열 블록(To·Ho)의 -10.9%도 실제 강성 저하가 아니라 가열에 따른 모드 전환으로 본다.

조건군	피크 주파수 중앙값	피크 이동률(대 101 Hz)	평균 이상 점수	임계값 초과 비율
정상 (Tx·Hx)	101.0 Hz	+0.0%	0.03	0/30 (0%)
습도 변화 (Tx·Ho)	101.0 Hz	+0.0%	0.07	0/30 (0%)
온습도 동시 (To·Ho)	90.0 Hz	-10.9%	4.13	4/30 (13.3%)
M500·B0	36.0 Hz	-64.4%	33.2	105/105 (100%)
M1000·B0	36.0 Hz	-64.4%	128.4	105/105 (100%)
M0·B1	64.0 Hz	-36.6%	264.7	105/105 (100%)
M0·B2	40.0 Hz	-60.4%	370.3	105/105 (100%)
M500·B1	44.0 Hz	-56.4%	75.4	105/105 (100%)
M1000·B1	81.0 Hz	-19.8%	108.3	105/105 (100%)
M500·B2	66.0 Hz	-34.7%	307.1	105/105 (100%)

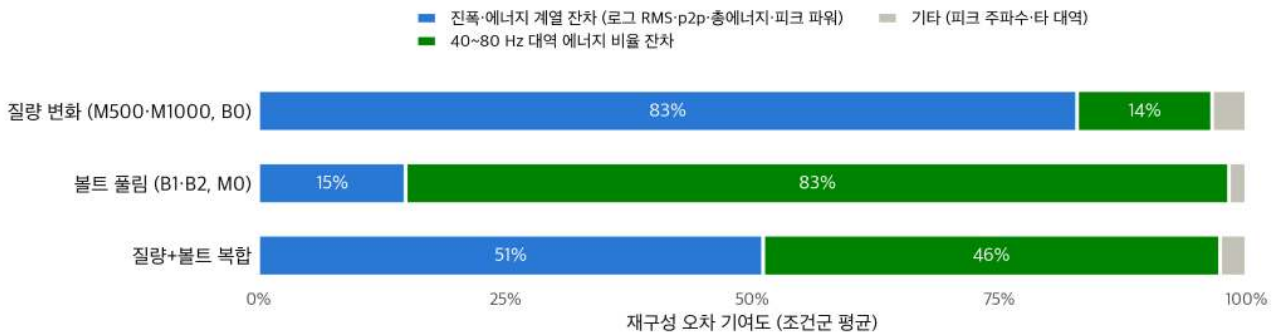
M1000·B2	53.0 Hz	-47.5%	323.4	105/105 (100%)
----------	---------	--------	-------	----------------

모델	오탐률	Precision	Recall	F1-score
무보정 AE (환경 입력 없음)	0.022	0.998	0.993	0.995
온도 보정 AE (T 잔차)	0.033	0.996	1.000	0.998
온도·습도 보정 AE (T·RH 잔차)	0.044	0.995	1.000	0.998

2차에서는 보정 모델의 오탐이 2건에서 4건으로 늘었으나 손상 재현율은 시드와 무관하게 100%로 유지되었다(해석은 3.5).

지표	무보정 AE	온습도 보정 AE
습도 변화 블록 오탐률 (Tx·Ho)	0.0% (0/30)	0.0% (0/30)
온습도 동시 블록 오탐률 (To·Ho)	6.7% (2/30)	13.3% (4/30)
손상 재현율 (840창, 시드 5개 범위)	96.2~99.3%	100% (전 시드)

To·Ho의 2건과 4건은 30창 중 두 건 차이로, 창 수가 적어 우연히도 생길 수 있는 수준이며, 보정 오탐 4건 중 3건은 배율 1.0~1.5배의 경계 초과다. 유형별 기여도는 그림5와 같다.



[그림5: 손상 유형별 재구성 오차 피쳐 기여도 — 질량형·볼트형·복합형 프로파일]

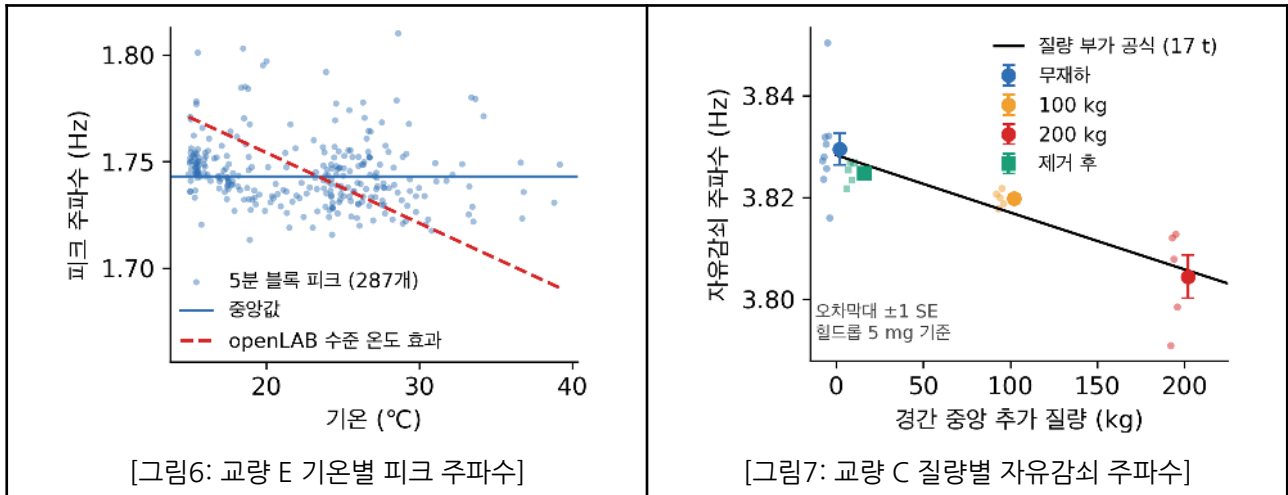
3차 실험 결과

교량 E에서는 4일간의 모든 세션에서 약 1.74 Hz 피크가 잡음과 뚜렷이 구분되어 안정적으로 추적되었고, 세션 간 차이는 1% 미만이었다. 한 세션의 임계값을 다른 세션에 적용한 무보정 오탐률은 재부착 뒤에도 대부분 2% 이하였다.

온습도 보정의 이득은 나타나지 않았다. 학습과 평가의 온도 범위가 겹칠 때는 네 모델의 오탐률이 약 1~2%로 같았고, 평가 온도가 학습 범위를 벗어나면 보정 모델의 오탐률이 오히려 커졌다. 같은 부착 구간(기온 약 15~39 °C)에서 통행 진폭을 통제하고 피크 주파수의 기온 기울기를 추정하니 0과 구분되지 않았다. openLAB 수준의 온도 효과(-0.19%/°C)였다면 24 °C 범위에서 피크가 약 4.5% 낮아졌어야 하지만 실측은 전 구간에서 평평했다(그림6). 실측 기울기로 본 피크 변화는 약 0.5%로 5분 블록 사이의 흔들림 수준이었다. 오히려 통행 진폭이 커질수록 피크가 조금 낮아지는 진폭 의존이 더 뚜렷했다.

초과 창이 출쳐도 온도가 아니었다. 초과는 통행이 적은 저녁~심야, 특히 진폭이 작은 창에 집중되었고, 세 축이 0으로 찍히는 센서 오류는 두 교량 모두에서 간헐적으로 발생했다(E 기록의 약 10%, 해당 창은 분석에서 제외). 교량 C의 예비 측정(9월 20일 같은 부착 3세션, 약 6시간)에서는 선별 측정에서 고른 3.81 Hz 모드 피크가 세션마다 재현되었고, 인접 피크가 함께 나타나 추적 규칙을 미리 고정했다. 예비 재하 100 kg의 주파수 하락은 측정 오차와 겨우 구분되는 수준이어서 본시험 질량을 200 kg으로 정했다. 200 kg을 경간 중앙에 올리자 주파수가

0.6% 낮아졌고 제거 후 원래 값으로 돌아왔으며, 진폭 차이를 보정하면 무재하 · 100 kg · 200 kg의 주파수는 오차 범위 안에서 하나의 질량 부가 곡선 위에 놓여 등가질량 약 17 t로 일관되었다(그림7). 무재하 걷기 20분으로 학습한 모델은 학습에 쓰지 않은 무재하 걷기 10분에서 창의 18%가 임계값을 넘었으나 3창 이상 연속된 초과는 없었고, 200 kg 재하 걷기 30분에서는 창의 46%가 넘고 약 32%가 연속 3창 경보로 이어졌다(최장 7창 연속). 단일 창 기준으로는 무재하에서도 18%가 넘어 구분이 약하지만 연속 초과 비율에서는 뚜렷이 갈려, 연속 창 판정이 오탐을 거르면서 상태 변화는 놓치지 않았다. 경보의 원인이 질량인지는 고유진동수의 하락·복귀로 가려냈다.



3.5 결과 분석

1차 결과는 예상한 물리 현상과 들어맞았다. 1-1의 피크 주파수 8.9% 하락은 질량이 늘면 고유진동수가 낮아진다($f = \frac{1}{2\pi} \sqrt{\frac{k}{m}}$)는 예상 그대로이고, 1-0은 진폭·대역 피치가 반응해 초과 비율이 4.5%(정상)에서 29.7%로 올라 여러 피처를 종합한 이상 점수의 필요성을 확인했다. 다만 폼보드의 감쇠와 수동 타격의 편차로 정상 측정값의 산포가 커서, 이 둘이 2차의 개선 목표가 되었다.

2차에서는 첫째, 가진을 기계로 표준화하자 정상 기준이 고정되었다. 상온 정상 평가창 60개의 피크가 주파수 분해능 안에서 모두 101 Hz로 일치했다. 둘째, 27조건이 사실상 완전히 구분되었다. 배울 중앙값이 정상 창은 0.003~0.038배, 손상 창은 5.8~73.7배로 백 배 넘게 벌어졌다. 다만 가열 블록의 정상 창 하나가 15.9배까지 튀었고 가장 약한 손상 창은 2.0배에 그쳐, 운용에서는 연속된 창의 초과 비율로 판단한다. 심각도 서열은 질량에서 M500 5.8배 → M1000 22.6배의 단조 증가가 세 블록 모두에서 성립했고, 볼트는 단독으로는 B1 → B2로 점수가 커졌으나 질량과 겹치면 서열이 뒤바뀌어(M500·B1 75.4 < M0·B1 264.7) 심각도 지표로 쓰기 어려웠다.

셋째, 재구성 오차의 기여도 구성이 손상의 종류를 가리킨다(그림5). 볼트 풀림 창에서는 40~80 Hz 대역 비율 간차가 76~90%를, 질량 창에서는 진폭·에너지 계열이 73~80%를 차지했고, 복합 창에서는 두 계열이 51%·46%로 나란히 나타났다. 대역 비율이 지배적이면 경계 조건(볼트)을, 진폭·에너지 계열이 지배적이면 하중 변화를 먼저 점검한다. 피크 주파수는 모드 전환으로 흔들려 피처 종합이 필요하다는 1차 결론이 유효했다.

넷째, 정상 학습 데이터가 온도 22.4~39.9 °C, 습도 48.9~79.0%를 미리 포괄해 보정 없이도 오탐이 드물었고, 보정의 효과는 재현율 유지(시드 5개 전부 100%, 무보정 96.2~99.3%)에 그쳤다. 남은 오탐 4건은 가장 뜨거웠던 39.9 °C 부근 창으로, 공기 온도로 구조물 온도를 근사한 한계다.

성능 개선의 가장 큰 몫은 피처 정규화다. 조건 사이 진폭이 약 40배까지 달라 원시 피처로는 재현율이 21.3%(질량 창 0%)에 그쳤지만, 로그·비율 정규화로 같은 데이터에서 100%가 되었다.

3차 결과가 보여주는 것은 네 가지다. 첫째, 시스템은 실교량으로 옮겨도 성립한다. 두 육교에서 모드 피크가 1% 이내로 재현되고 오탐도 낮아, 노드를 옮겨 다니는 방문형 점검이 가능함을 보였다. 둘째, 선행연구[2]의 계절 오탐 급증이 교량 E에서 재현되지 않은 것은 조건이 달랐기 때문이다. Z24는 영하 구간을 포함한 1년 관측에서 포장층이 얼며 크게 오른 고유진동수를 직접 판정에 썼다. 반면 교량 E는 환경과 판정 방식이 모두 달랐다. 환경 면에서는 영하 없이 15 °C 이상인 초가을 4일이라 온도에 의한 피크 변화가 약 0.5%로 블록 간 흔들림 수준에 그쳤다(3.4). 판정 면에서는 오토인코더가 주파수를 직접 보지 않고 8개 피치를 함께 본 점수를 99% 분위수 임계값과 비교하므로, 수 % 주파수 이동이 점수에 거의 드러나지 않는다. Z24와 조건이 비슷한 openLAB의 저온 외삽에서는 주파수 채널을 결합하자 급증이 재현되고 보정으로 해소되었다(2.7). 즉 보정은 환경 변화가 피쳐 흔들림보다 크고 판정이 주파수를 직접 볼 때 필요하며, 학습 범위 밖 입력은 외삽 오차를 더하므로 표시만 해야 한다. 셋째, 실교량 오탐의 원인은 온도가 아니라 심야 저진폭 같은 가진 조건이었다. 교량 E의 초과 창은 진폭·에너지 계열이 지배해 기여도도 가진 변화를 가리켰고, 교량 C의 재하 구간은 세 계열이 고르게 나뉘어 기여도만으로는 원인을 특정할 수 없었다. 넷째, 두 재하(100kg, 200kg)의 주파수 하락·복귀가 질량 부가 공식과 맞아, 고유진동수 추적이 경보의 원인을 가리는 근거가 된다.

4 해석 & 결론

4.1 결론

본 시스템은 저비용 상용 부품(ADXL355Z·DHT22·Raspberry Pi)만으로 실험실 모형의 27조건 매트릭스를 구분했다. 온습도 보정 모델은 손상 840창을 모두 검출하고 정상 평가 90창 중 4창에서만 오탐을 내(재현율 100%·오탐률 4.4%·ROC-AUC 0.996), 재현율 0.234에 그쳤던 1차의 검출 누락을 가진 표준화와 피쳐 정규화로 해소했다.

환경 보정은 학습이 환경 변동을 포괄하지 못한 경우(1차 습도 변화 구간 17.5% → 9.1%, 주파수 채널을 결합한 openLAB 계절 외삽 26% → 0~3%)에 오탐을 줄였고 효과는 주로 온도에서 나왔으며, 환경 범위를 미리 포괄한 2차에서는 재현율 유지로만 나타났다. 기여도 분석에서 볼트 풀림은 40~80 Hz 대역 비율 잔차가, 질량 변화는 진폭·에너지 계열 잔차가 지배해 알람만으로 점검 방향을 좁혔다.

3차 실교량에서는 두 육교의 모드 피크를 1% 이내로 추적했고 오탐도 낮았으나, 초가을 현상은 피크의 온도 의존이 잡음 아래여서 보정 이득은 확인되지 않았고 오탐은 가진 부족과 통행 사건에서 나왔다. 추가질량 실험에서는 200 kg의 0.6% 주파수 하락·복귀를 측정해, 단일 저비용 노드로 모드 등가질량(약 17 t)의 약 1% 수준 질량 변화를 가려냈다. 이 결과에 따라 현장 운용은 120초 창과 연속 3창 초과 규칙으로 판정하고, 통행이 없는 저진폭 구간은 판정에서 제외하며, 센서 오류가 수 초 이상 이어지면 자동 재초기화하도록 설정한다.

4.2 추후 개선사항

추후 과제는 네 가지다. 첫째, 실교량에서의 기여도 표기를 확정한다. 실교량에는 손상 라벨이 없고 진폭·에너지 계열은 통행량에 좌우되므로 2차의 대응 관계를 그대로 쓰지 않고, 정상 기간 대비 변화량을 피쳐 계열(진폭·대역 구조·주파수) 단위로 보고한다. 원인은 점검 시작점 후보로만 제시하고 위치·정도는 판단하지 않으며(Rytter의 손상 식별 4단계 중 1단계인 존재 탐지[9]), 후보별 기여 프로파일은 손상 이력이 기록된 Z24 자료[3]로 먼저 검증한다. 둘째는 온습도 보정 효과의 재검증으로, 영하 구간과 계절 전이를 포함하도록 같은 부착으로 수 주 이상 관측하고 학습일과 평가일을 날짜 단위로 나누며, 기온 센서 차광과 표면온도 센서 밀착으로 환경 입력을 확보한다. 셋째, 주파수 판정 채널(2.7)을 실시간 시스템에 통합한다. 넷째, 다중 노드로 확장해 손상 위치를 추정한다.

참고문헌

- [1] Caballero-Russi, D., Ortiz, A. R., Guzmán, A., & Canchila, C., “Design and Validation of a Low-Cost Structural Health Monitoring System for Dynamic Characterization of Structures,” Applied Sciences, 12(6), 2807, 2022.
- [2] Peeters, B., & De Roeck, G., “One-year monitoring of the Z24-Bridge: environmental effects versus damage events,” Earthquake Engineering & Structural Dynamics, 30(2), 149-171, 2001.
- [3] Z24 Bridge Benchmark documentation, KU Leuven, Leuven, Belgium.
- [4] 한국표준과학연구원(KRISS), BOCDA(Brillouin Optical Correlation Domain Analysis) 기반 분포형 광섬유 센서 기술.
- [5] Giglioni, V., Venanzi, I., Baia, A. E., Poggioni, V., Milani, A., & Ubertini, F., “Deep Autoencoders for Unsupervised Damage Detection with Application to the Z24 Benchmark Bridge,” European Workshop on Structural Health Monitoring (EWSHM 2022), Lecture Notes in Civil Engineering, Springer, 2023. DOI: 10.1007/978-3-031-07258-1_105
- [6] Pakzad, S. S., & Masoodi, A. R., “Early damage detection in bridges using a variational autoencoder-based hybrid unsupervised learning framework,” Scientific Reports, 16, 1389, 2026. DOI: 10.1038/s41598-025-31115-w
- [7] Aosong Electronics, DHT22(AM2302) 온습도 센서 Datasheet.
- [8] Jansen, A., Richter, B., Röder, R., Herbers, M., & Marx, S., "Monitoring Data of the openLAB Research Bridge (2024-02-01 to 2024-10-31) and building information," OPARA - Open Access Repository and Archive, TU Dresden, 2025. DOI: 10.25532/OPARA-660.
- [9] Rytter, A., "Vibrational Based Inspection of Civil Engineering Structures," PhD thesis, Aalborg University, Denmark, 1993.

자료 목록

번호	자료형태	본문 표기	자료 설명(키워드)	비고(출처 등)
1	사진	자료1	정자교 붕괴 현장 사진	안전저널 보도(2023.4.7.), https://www.anjunj.com/news/articleView.html?idxno=36589
2	사진	자료2	보릿돌교 붕괴 사진(CCTV 화면)	연합뉴스 보도(2026.7.28.) 화면 캡처, https://v.daum.net/v/20260728161255607
3	차트	자료3	전국 교량 중 제3종 교량의 비율	국토안전관리원 2025년 시설물 종별 현황 자료 기반 직접 작성
4	표	자료4	시설물 등급별 점검 주기 및 대상 표	「시설물의 안전 및 유지관리에 관한 특별법 시행령」 별표 3(2025.12.4 개정) 기반 직접 작성

5	도식·그래프	그림1~그림7	배치 개념도, Autoencoder 구조, 시스템 순서도, 실험 결과 그래프	직접 제작
6	사진	사진 일체	모형 교량, 완성 모듈, 2차 모형 제작 과정 사진	직접 촬영
7	공개 데이터	데이터셋	openLAB(TU Dresden, OPARA — 2.7 분석에 사용), Z24 Bridge Benchmark(KU Leuven — 선행 연구 비교 참조)	공개 데이터, 참고문헌 3·8
8	오픈소스 SW	-	Python, NumPy, pandas, scikit-learn, Matplotlib (수집·분석·시각화)	각 공식 배포본(PSF·BSD 라이선스)

소스코드

sentry.py

```

from __future__ import annotations
import argparse
import csv
import io
import json
import math
import socket
import re
import tempfile
import subprocess
import sys
import threading
import time
from collections import deque
from pathlib import Path

import numpy as np
import pandas as pd
from flask import Flask, jsonify

HTML = r'''<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>SENTRY 보안 대시보드</title>
<style>
body{margin:0;background:#10141b;color:#eef2f7;font-family:Arial,sans-serif}.wrap{
max-width:1100px;margin:auto;padding:20px}.top{display:flex;justify-content:space-
between;align-items:center;gap:16px;margin-bottom:16px}.title{font-size:28px;font-
weight:700}.small{font-size:12px;color:#9ca8b8;margin-top:7px}.status{padding:11px
18px;border-radius:999px;font-weight:700;white-space:nowrap}.normal{background:#17
6b45}.warning{background:#916b12}.anomaly{background:#9d2c2c}.waiting{background:#
465164}.grid{display:grid;grid-template-columns:repeat(6,1fr);gap:12px}.card{backg
round:#1a202b;border-radius:14px;padding:16px;box-shadow:0 6px 18px
#0004}.full{grid-column:span 6}.wide{grid-column:span

```

```

3}.label{font-size:13px;color:#9ca8b8}.value{font-size:27px;font-weight:700;margin-top:6px}.buttons{display:flex;gap:10px;flex-wrap:wrap}.button{border:0;border-radius:10px;padding:11px 15px;font-weight:700;color:white;background:#2f73bf;cursor:pointer}.button.warn{background:#8b6518}.button.stop{background:#6b3440}.progress{height:10px;background:#111722;border-radius:999px;overflow:hidden;margin-top:12px}.progress>div{height:100%;width:0;background:#6db7ff}.alarmbox{border:1px solid #344052}.alarmbox.active{border-color:#d64b4b;box-shadow:0 0 28px #d64b4b55}canvas{width:100%;height:280px;background:#111722;border-radius:10px}.thresholds{display:grid;grid-template-columns:repeat(3,1fr);gap:12px;margin-top:12px}.threshold{background:#111722;border-radius:10px;padding:12px}.reason{font-size:18px;font-weight:700;margin-top:8px}@media(max-width:800px){.grid{grid-template-columns:repeat(2,1fr)}.wide,.full{grid-column:span 2}.top{align-items:flex-start;flex-direction:column}.thresholds{grid-template-columns:1fr}}
</style>
</head>
<body>
<div class="wrap">
<div class="top"><div><div class="title">SENTRY 기체 보안 대시보드</div><div id="connection" class="small">서버 연결 확인 중</div></div><div id="securityStatus" class="status waiting">감시 해제</div></div>
<div class="grid">
<div class="card"><div class="label">진동 RMS</div><div id="vibration" class="value">-</div><div id="vibrationRatio" class="small">-</div></div>
<div class="card"><div class="label">순간 피크</div><div id="peakMotion" class="value">-</div><div id="peakRatio" class="small">-</div></div>
<div class="card"><div class="label">자세 변화</div><div id="tilt" class="value">-</div><div id="tiltThreshold" class="small">-</div></div>
<div class="card"><div class="label">표면온도 · DS18B20</div><div id="temp" class="value">-</div><div id="surfaceState" class="small">대기</div></div>
<div class="card"><div class="label">습도 · DHT22</div><div id="hum" class="value">-</div><div id="humidityState" class="small">대기</div></div>
<div class="card"><div class="label">누적 경보</div><div id="alertCount" class="value">0</div></div>
<div class="card full alarmbox" id="alarmBox"><div class="label">최근 경보</div><div id="reason" class="reason">없음</div><div id="lastAlert" class="small">-</div></div>
<div class="card full"><div class="label">CSV 저장 상태</div><div id="storage" class="small">대기</div></div>
<div class="card full"><div class="label">최근 3축 합성 진동</div><canvas id="wave"></canvas></div>
<div class="card full"><div class="label">감시 기준</div><div class="thresholds"><div class="threshold"><div class="label">진동 기준</div><div id="vibrationThreshold" class="value">-</div></div><div class="threshold"><div class="label">충격 기준</div><div id="peakThreshold" class="value">-</div></div><div class="threshold"><div class="label">자세 기준</div><div id="angleThreshold" class="value">-</div></div></div><div class="progress"><div id="calibrationBar"></div></div><div id="calibrationText" class="small">기체를 설치한 뒤 감시 시작을 누른다.</div></div>
<div class="card full"><div class="buttons"><button class="button" onclick="action('/api/security/arm')">감시 시작 · 기준 재설정</button><button class="button warn" onclick="action('/api/security/reset')">경보 해제</button><button class="button stop" onclick="action('/api/security/disarm')">감시 해제</button></div><div id="info" class="small">-</div></div>
</div>
</div>
<script>
let audioContext=null,lastAlert=false,lastConnected=true,lastArmed=false;
function
enableAudio(){if(!audioContext)audioContext=new(window.AudioContext||window.webkitAudioContext)();if(audioContext.state==='suspended')audioContext.resume();if('Noti

```

```

fication'in
window&&Notification.permission==='default')Notification.requestPermission() }
function beep() {if(!audioContext)return;const
o=audioContext.createOscillator(),g=audioContext.createGain();o.frequency.value=88
0;g.gain.setValueAtTime(.16,audioContext.currentTime);g.gain.exponentialRampToValu
eAtTime(.001,audioContext.currentTime+.28);o.connect(g);g.connect(audioContext.des
tination);o.start();o.stop(audioContext.currentTime+.3)}
function notify(title,body){if('Notification'in
window&&Notification.permission==='granted')new Notification(title,{body})}
function fmt(v,d,u){return v==null?'-':Number(v).toFixed(d)+u}
function line(canvas,x,y){const
c=document.getElementById(canvas),d=devicePixelRatio||1,w=c.clientWidth,h=c.client
Height;c.width=w*d;c.height=h*d;const
g=c.getContext('2d');g.scale(d,d);g.clearRect(0,0,w,h);g.strokeStyle='#2c3544';g.b
eginPath();for(let i=1;i<5;i++){const
yy=h*i/5;g.moveTo(0,yy);g.lineTo(w,yy)}g.stroke();if(!y||y.length<2)return;let
ymin=Math.min(...y),ymax=Math.max(...y);if(ymax===ymin){ymax+=1;ymin-=1}g.strokeSt
yle='#67b2ff';g.lineWidth=2;g.beginPath();for(let i=0;i<y.length;i++){const
px=i/(y.length-1)*w,py=h-(y[i]-ymin)/(ymax-ymin)*h;if(i===0)g.moveTo(px,py);else
g.lineTo(px,py)}g.stroke()}
async function action(path){enableAudio();try{await
fetch(path,{method:'POST'})};await update()}catch(e){}}
async function update(){const conn=document.getElementById('connection');try{const
r=await fetch('/api',{cache:'no-store'});if(!r.ok)throw new Error('api');const
s=await r.json();const
st=document.getElementById('securityStatus');st.textContent=s.security_status_text
||'대기 중';st.className='status
'+(s.security_status_class||'waiting');document.getElementById('vibration').textCo
ntent=fmt(s.vibration_rms_g,4,'
g');document.getElementById('peakMotion').textContent=fmt(s.vibration_peak_g,4,'
g');document.getElementById('tilt').textContent=fmt(s.orientation_change_deg,1,'°'
);document.getElementById('tiltThreshold').textContent=s.orientation_threshold_deg
==null?'-': '기준
'+s.orientation_threshold_deg.toFixed(1)+'°';document.getElementById('temp').textC
ontent=fmt(s.surface_temp_c,1,'
°C');document.getElementById('hum').textContent=fmt(s.humidity,1,'
%');document.getElementById('surfaceState').textContent=s.surface_status||'대기';do
cument.getElementById('humidityState').textContent=s.humidity_status||'대기';docume
nt.getElementById('storage').textContent=(s.logger_status||'')+ '
'+(s.storage_status||'')+ '
'+(s.active_csv||'파일 대기')+ '
'+(s.run_folder||'');document.getElementById('alertCount').textContent=String(s.al
ert_count||0);document.getElementById('vibrationThreshold').textContent=fmt(s.vibr
ation_threshold_g,4,'
g');document.getElementById('peakThreshold').textContent=fmt(s.peak_threshold_g,4,'
g');document.getElementById('angleThreshold').textContent=fmt(s.orientation_thresh
old_deg,1,'°');document.getElementById('vibrationRatio').textContent=s.vibration_r
atio==null?'-': '기준의
'+s.vibration_ratio.toFixed(2)+'배';document.getElementById('peakRatio').textConte
nt=s.peak_ratio==null?'-': '기준의 '+s.peak_ratio.toFixed(2)+'배';const
progress=Math.max(0,Math.min(100,s.calibration_progress||0));document.getElementBy
Id('calibrationBar').style.width=progress+'%';document.getElementById('calibration
Text').textContent=s.calibration_text||'-';document.getElementById('reason').textC
ontent=s.last_alert_reason||'없음';document.getElementById('lastAlert').textContent
=s.last_alert_at_text||'-';document.getElementById('info').textContent=s.info||'-';
document.getElementById('alarmBox').className='card full
alarmbox'+(s.security_alert?'
active':'');line('wave',s.wave_x,s.wave_y);if(s.updated_at===null){conn.textContent
='서버 연결됨 · 센서 데이터 대기'}else{const
age=Math.max(0,Date.now()/1000-s.updated_at);conn.textContent=age<3?'서버 연결됨 ·
실시간 감시 중:'서버 연결됨 · 마지막 센서 데이터 '+age.toFixed(1)+'초
전'}if(s.security_alert&&!lastAlert){beep();setTimeout(beep,700);notify('SENTRY
도난·이동 의심',s.last_alert_reason||'기체 움직임이
감지됨')}}lastAlert=!!s.security_alert;lastConnected=true;lastArmed=!!s.security_arm

```

```

ed}catch(e){conn.textContent='서버 연결 끊김';const
st=document.getElementById('securityStatus');st.textContent='연결
끊김';st.className='status
anomaly';if(lastConnected&&lastArmed){beep();notify('SENTRY 연결 끊김','장치 전원
또는 네트워크 상태를 확인해야 한다.')}lastConnected=false}}
setInterval(update,500);update();
</script>
</body>
</html>'''

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="SENTRY continuous security
dashboard")
    parser.add_argument("--logger", default="logger.py")
    source = parser.add_mutually_exclusive_group()
    source.add_argument("--bridge", help="기존 CSV 한 개 열기(자동 파일 전환 없음)")
    source.add_argument("--manifest", help="실행 폴더의 current.json을 따라가기")
    parser.add_argument("--condition", default="SECURITY")
    parser.add_argument("--output-dir", default="data/security")
    parser.add_argument("--sample-rate", type=int, choices=[125, 250, 500],
default=125)
    parser.add_argument("--range-g", type=int, choices=[2, 4, 8], default=8)
    parser.add_argument("--no-dht", action="store_true")
    parser.add_argument("--no-surface", action="store_true")
    parser.add_argument("--surface-device", help="DS18B20 장치 ID: 28-...")
    parser.add_argument("--rotate-seconds", type=float, default=3600.0)
    parser.add_argument("--flush-seconds", type=float, default=1.0)
    parser.add_argument("--fsync-seconds", type=float, default=10.0)
    parser.add_argument("--env-stale-seconds", type=float, default=10.0)
    parser.add_argument("--host", default="0.0.0.0")
    parser.add_argument("--port", type=int, default=8080)
    parser.add_argument("--calibration-seconds", type=float, default=20.0)
    parser.add_argument("--window-seconds", type=float, default=0.5)
    parser.add_argument("--step-seconds", type=float, default=0.25)
    parser.add_argument("--vibration-multiplier", type=float, default=4.0)
    parser.add_argument("--peak-multiplier", type=float, default=4.0)
    parser.add_argument("--min-vibration-g", type=float, default=0.015)
    parser.add_argument("--min-peak-g", type=float, default=0.08)
    parser.add_argument("--tilt-deg", type=float, default=8.0)
    parser.add_argument("--tilt-hold-seconds", type=float, default=1.0)
    parser.add_argument("--vote-window", type=int, default=5)
    parser.add_argument("--required-votes", type=int, default=3)
    parser.add_argument("--stale-seconds", type=float, default=5.0)
    parser.add_argument("--auto-arm", action="store_true")
    args = parser.parse_args()
    if not re.fullmatch(r"[\w.-]+", args.condition) or args.condition in {".",
".."}:
        parser.error("condition에는 문자, 숫자, _, -, .만 사용할 수 있습니다.")
    for name in ("rotate_seconds", "flush_seconds", "fsync_seconds",
"env_stale_seconds"):
        if not math.isfinite(getattr(args, name)) or getattr(args, name) <= 0:
            parser.error(f"{name}은 유효한 양수여야 합니다.")
    return args

def metadata_path(bridge: Path) -> Path:
    if not bridge.name.endswith("_bridge_log.csv"):
        raise ValueError("bridge CSV 이름은 *_bridge_log.csv 형식이어야 합니다.")
    return bridge.with_name(bridge.name[:-len("_bridge_log.csv")] +
"_metadata.json")

def metadata_for(bridge: Path) -> dict:

```

```

if bridge.suffix.lower() == ".json":
    return json.loads(bridge.read_text(encoding="utf-8"))
path = metadata_path(bridge)
if path.exists():
    return json.loads(path.read_text(encoding="utf-8"))
return {"sample_rate_hz": 125, "range_g": 8}

def tail_dataframe(path: Path, rows: int) -> pd.DataFrame:
    # CSV 전체를 읽지 않는다. 쓰기 중인 마지막 불완전 행은 다음 조회로 미룬다.
    with path.open("rb") as file:
        header = file.readline().decode("utf-8").strip()
        file.seek(0, 2)
        position = file.tell()
        blocks, count = [], 0
        while position > 0 and count <= rows + 1:
            size = min(65536, position)
            position -= size
            file.seek(position)
            block = file.read(size)
            blocks.append(block)
            count += block.count(b"\n")
        truncated = position > 0
        raw = b"".join(reversed(blocks))
    # 마지막 \n까지의 완성된 레코드만 사용한다.
    raw = raw[:raw.rfind(b"\n") + 1]
    lines = raw.decode("utf-8", errors="ignore").splitlines()
    if lines and lines[0] == header:
        lines = lines[1:]
    elif truncated and lines:
        lines = lines[1:]
    lines = lines[-rows:]
    if not lines or not header:
        return pd.DataFrame()
    return pd.read_csv(io.StringIO(header + "\n" + "\n".join(lines)))

def manifest_csv(source: Path, filename: str | None) -> Path | None:
    if filename is None:
        return None
    if Path(filename).name != filename or filename in {".", ".."}:
        raise ValueError("current.json의 CSV 이름이 올바르지 않습니다.")
    return source.parent / filename

def read_live_tail(source: Path, rows: int) -> tuple[pd.DataFrame, dict, Path]:
    metadata = metadata_for(source)
    if source.suffix.lower() != ".json":
        return tail_dataframe(source, rows), metadata, source
    current = manifest_csv(source, metadata.get("current_csv"))
    if current is None:
        raise ValueError("current.json에 current_csv가 없습니다.")
    data = tail_dataframe(current, rows)
    previous = manifest_csv(source, metadata.get("previous_csv"))
    if len(data) < rows and previous is not None and previous.exists():
        older = tail_dataframe(previous, rows - len(data))
        frames = [frame for frame in (older, data) if not frame.empty]
        data = pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()
    return data.tail(rows), metadata, current

def last_number(data: pd.DataFrame, column: str) -> float | None:
    # dropna().iloc[-1]는 지난 파일/지난 시각의 값을 되살리므로 사용하지 않는다.
    if data.empty or column not in data:

```

```

        return None
    value = pd.to_numeric(data[column].iloc[-1], errors="coerce")
    return finite_or_none(value)

def finite_or_none(value: float | None) -> float | None:
    if value is None:
        return None
    number = float(value)
    return number if math.isfinite(number) else None

def format_alert_time(value: float | None) -> str | None:
    if value is None:
        return None
    return time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(value))

def security_window_metrics(values: np.ndarray) -> tuple[float, float, np.ndarray, np.ndarray]:
    center = np.median(values, axis=0)
    dynamic = values - center
    magnitude = np.linalg.norm(dynamic, axis=1)
    rms = float(np.sqrt(np.mean(magnitude * magnitude)))
    peak = float(np.max(magnitude))
    return rms, peak, center, magnitude

class SecurityMonitor:
    def __init__(self, bridge: Path, args: argparse.Namespace):
        self.bridge = bridge # JSON manifest 또는 단일 CSV
        self.logger_process = getattr(args, "logger_process", None)
        self.env_stale_seconds = float(metadata_for(bridge).get("env_stale_seconds", args.env_stale_seconds))
        self.recent_data = pd.DataFrame()
        self.fs = float(metadata_for(bridge).get("sample_rate_hz", args.sample_rate))
        self.calibration_seconds = max(5.0, args.calibration_seconds)
        self.window_seconds = max(0.2, args.window_seconds)
        self.step_seconds = max(0.1, args.step_seconds)
        self.vibration_multiplier = max(1.0, args.vibration_multiplier)
        self.peak_multiplier = max(1.0, args.peak_multiplier)
        self.min_vibration_g = max(0.0001, args.min_vibration_g)
        self.min_peak_g = max(0.0001, args.min_peak_g)
        self.orientation_threshold_deg = max(1.0, args.tilt_deg)
        self.orientation_hold_seconds = max(0.2, args.tilt_hold_seconds)
        self.vote_window = max(1, args.vote_window)
        self.required_votes = min(max(1, args.required_votes), self.vote_window)
        self.stale_seconds = max(2.0, args.stale_seconds)
        self.lock = threading.Lock()
        self.baseline_vector: np.ndarray | None = None
        self.vibration_threshold_g: float | None = None
        self.peak_threshold_g: float | None = None
        self.calibration_start_ns: float | None = None
        self.calibration_requested = False
        self.armed = False
        self.alert_latched = False
        self.alert_count = 0
        self.last_alert_at: float | None = None
        self.last_alert_reason: str | None = None
        self.last_evaluated_ns = 0.0
        self.tilt_started_ns: float | None = None
        self.motion_votes: deque[bool] = deque(maxlen=self.vote_window)
        self.alert_path = (

```

```

        bridge.parent / "security_alerts.csv" if bridge.suffix.lower() ==
".json"
        else bridge.with_name(bridge.name.replace("_bridge_log.csv",
"_security_alerts.csv"))
    )
    self.state = {
        "security_armed": False,
        "security_alert": False,
        "security_status_text": "감시 해제",
        "security_status_class": "waiting",
        "vibration_rms_g": None,
        "vibration_peak_g": None,
        "orientation_change_deg": None,
        "vibration_threshold_g": None,
        "peak_threshold_g": None,
        "orientation_threshold_deg": self.orientation_threshold_deg,
        "vibration_ratio": None,
        "peak_ratio": None,
        "temp_c": None, # 참고용 DHT22 기온. 화면의 주 온도는 표면온도이다.
        "surface_temp_c": None,
        "surface_age_s": None,
        "dht_age_s": None,
        "humidity": None,
        "surface_enabled": True,
        "dht_enabled": True,
        "active_csv": "",
        "run_folder": str(bridge.parent),
        "storage_status": "waiting",
        "rotate_seconds": args.rotate_seconds,
        "alert_count": 0,
        "last_alert_at": None,
        "last_alert_at_text": None,
        "last_alert_reason": None,
        "calibration_progress": 0.0,
        "calibration_text": "기체를 설치한 뒤 감시 시작을 누른다.",
        "wave_x": [],
        "wave_y": [],
        "updated_at": None,
        "info": "센서 데이터를 기다리는 중",
    }
    threading.Thread(target=self.loop, daemon=True).start()
    if args.auto_arm:
        self.arm()

def arm(self) -> dict:
    with self.lock:
        self.calibration_requested = True
        self.calibration_start_ns = None
        self.baseline_vector = None
        self.vibration_threshold_g = None
        self.peak_threshold_g = None
        self.armed = False
        self.alert_latched = False
        self.last_evaluated_ns = 0.0
        self.motion_votes.clear()
        self.tilt_started_ns = None
        self.state.update(
            {
                "security_armed": False,
                "security_alert": False,
                "security_status_text": "보정 중",
                "security_status_class": "warning",
                "calibration_progress": 0.0,
            }
        )

```

```

        "calibration_text": f"{self.calibration_seconds:.0f}초 동안
기체를 건드리지 않는다.",
        "vibration_rms_g": None,
        "vibration_peak_g": None,
        "orientation_change_deg": None,
        "vibration_ratio": None,
        "peak_ratio": None,
    }
)
return dict(self.state)

def disarm(self) -> dict:
    with self.lock:
        self.calibration_requested = False
        self.armed = False
        self.alert_latched = False
        self.motion_votes.clear()
        self.tilt_started_ns = None
        self.state.update(
            {
                "security_armed": False,
                "security_alert": False,
                "security_status_text": "감시 해제",
                "security_status_class": "waiting",
                "calibration_progress": 0.0,
                "calibration_text": "감시가 해제되었다.",
            }
        )
    return dict(self.state)

def reset_alert(self) -> dict:
    with self.lock:
        self.alert_latched = False
        self.motion_votes.clear()
        self.tilt_started_ns = None
        if self.armed:
            text, style = "감시 중", "normal"
        elif self.calibration_requested:
            text, style = "보정 중", "warning"
        else:
            text, style = "감시 해제", "waiting"
        self.state.update(
            {
                "security_alert": False,
                "security_status_text": text,
                "security_status_class": style,
            }
        )
    return dict(self.state)

def calibrate(self, data: pd.DataFrame) -> None:
    values = data[["ax_g", "ay_g", "az_g"]].to_numpy(float)
    window_samples = max(16, int(self.fs * self.window_seconds))
    if len(values) < window_samples * 4:
        return
    rms_values = []
    peak_values = []
    for start in range(0, len(values) - window_samples + 1, window_samples):
        rms, peak, _, _ = security_window_metrics(values[start:start +
window_samples])
        rms_values.append(rms)
        peak_values.append(peak)
    if len(rms_values) < 4:
        return

```

```

        baseline = np.median(values, axis=0)
        vibration_threshold = max(self.min_vibration_g,
float(np.quantile(rms_values, 0.99)) * self.vibration_multiplier)
        peak_threshold = max(self.min_peak_g, float(np.quantile(peak_values,
0.99)) * self.peak_multiplier)
        with self.lock:
            self.baseline_vector = baseline
            self.vibration_threshold_g = vibration_threshold
            self.peak_threshold_g = peak_threshold
            self.calibration_requested = False
            self.armed = True
            self.alert_latched = False
            self.motion_votes.clear()
            self.state.update(
                {
                    "security_armed": True,
                    "security_alert": False,
                    "security_status_text": "감시 중",
                    "security_status_class": "normal",
                    "vibration_threshold_g": vibration_threshold,
                    "peak_threshold_g": peak_threshold,
                    "calibration_progress": 100.0,
                    "calibration_text": "기준 측정이 끝났으며 보안 감시가 시작되었다.",
                    "info": f"기준 RMS {np.median(rms_values):.5f} g | 기준 피크
{np.median(peak_values):.5f} g | 최근 {self.vote_window}개 중
{self.required_votes}개 초과 시 경보",
                }
            )

    def write_alert(self, reason: str, now: float, metrics: dict) -> None:
        new_file = not self.alert_path.exists()
        with self.alert_path.open("a", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            if new_file:
                writer.writerow(["t_unix_s", "time", "reason", "vibration_rms_g",
"vibration_peak_g", "orientation_change_deg"])
            writer.writerow(
                [
                    f"{now:.6f}",
                    format_alert_time(now),
                    reason,
                    f"{metrics['vibration_rms_g']:.8f}",
                    f"{metrics['vibration_peak_g']:.8f}",
                    f"{metrics['orientation_change_deg']:.4f}",
                ]
            )

    def trigger_alert(self, reason: str, metrics: dict) -> None:
        with self.lock:
            if self.alert_latched:
                return
            now = time.time()
            self.alert_latched = True
            self.alert_count += 1
            self.last_alert_at = now
            self.last_alert_reason = reason
            self.state.update(
                {
                    "security_alert": True,
                    "security_status_text": "도난·이동 의심",
                    "security_status_class": "anomaly",
                    "alert_count": self.alert_count,
                    "last_alert_at": now,
                    "last_alert_at_text": format_alert_time(now),

```

```

        "last_alert_reason": reason,
    }
)
self.write_alert(reason, now, metrics)

def evaluate(self, data: pd.DataFrame, max_t: float) -> None:
    with self.lock:
        baseline = None if self.baseline_vector is None else
self.baseline_vector.copy()
        vibration_threshold = self.vibration_threshold_g
        peak_threshold = self.peak_threshold_g
        armed = self.armed
        if not armed or baseline is None or vibration_threshold is None or
peak_threshold is None:
            return
        segment = data[data["t_perf_ns"] >= max_t - self.window_seconds * 1e9]
        if len(segment) < max(16, int(self.fs * self.window_seconds * 0.7)):
            return
        values = segment[["ax_g", "ay_g", "az_g"]].to_numpy(float)
        vibration_rms, vibration_peak, current_vector, magnitude =
security_window_metrics(values)
        baseline_norm = float(np.linalg.norm(baseline))
        current_norm = float(np.linalg.norm(current_vector))
        angle = 0.0
        if baseline_norm > 1e-6 and current_norm > 1e-6:
            cosine = float(np.dot(baseline, current_vector) / (baseline_norm *
current_norm))
            angle = math.degrees(math.acos(max(-1.0, min(1.0, cosine))))
        vibration_exceeded = vibration_rms >= vibration_threshold
        peak_exceeded = vibration_peak >= peak_threshold
        motion = vibration_exceeded or peak_exceeded
        self.motion_votes.append(motion)
        votes = sum(self.motion_votes)
        if angle >= self.orientation_threshold_deg:
            if self.tilt_started_ns is None:
                self.tilt_started_ns = max_t
            else:
                self.tilt_started_ns = None
            tilt_sustained = self.tilt_started_ns is not None and (max_t -
self.tilt_started_ns) / 1e9 >= self.orientation_hold_seconds
            reason = None
            if vibration_peak >= peak_threshold * 2.0:
                reason = "강한 충격이 감지됨"
            elif tilt_sustained:
                reason = "기체 자세 변화가 지속됨"
            elif motion and angle >= self.orientation_threshold_deg * 0.5:
                reason = "진동과 자세 변화가 함께 감지됨"
            elif votes >= self.required_votes:
                reason = "진동이 반복적으로 감지됨"
            vibration_ratio = vibration_rms / vibration_threshold
            peak_ratio = vibration_peak / peak_threshold
            recent_t = segment["t_perf_ns"].to_numpy(float)
            metrics = {
                "vibration_rms_g": vibration_rms,
                "vibration_peak_g": vibration_peak,
                "orientation_change_deg": angle,
            }
        with self.lock:
            if self.alert_latched:
                text, style = "도난·이동 의심", "anomaly"
            elif motion or angle >= self.orientation_threshold_deg:
                text, style = "진동 감지", "warning"
            else:
                text, style = "감시 중", "normal"

```

```

self.state.update(
    {
        "security_armed": True,
        "security_alert": self.alert_latched,
        "security_status_text": text,
        "security_status_class": style,
        "vibration_rms_g": vibration_rms,
        "vibration_peak_g": vibration_peak,
        "orientation_change_deg": angle,
        "vibration_threshold_g": vibration_threshold,
        "peak_threshold_g": peak_threshold,
        "orientation_threshold_deg": self.orientation_threshold_deg,
        "vibration_ratio": vibration_ratio,
        "peak_ratio": peak_ratio,
        "wave_x": ((recent_t - recent_t[0]) / 1e9).tolist(),
        "wave_y": magnitude.tolist(),
        "info": f"최근 {self.vote_window}개 판정 중 움직임 {votes}개 |
자세 변화 {angle:.1f}° | {self.alert_path.name}",
    }
)
if reason is not None:
    self.trigger_alert(reason, metrics)

def loop(self) -> None:
    rows = int(self.fs * max(self.calibration_seconds + 5.0, 20.0))
    while True:
        try:
            if not self.bridge.exists():
                time.sleep(0.3)
                continue
            data, metadata, active_csv = read_live_tail(self.bridge, rows)
            with self.lock:
                self.state.update({
                    "active_csv": active_csv.name,
                    "storage_status": metadata.get("status", "CSV 직접 보기"),
                    "rotate_seconds": metadata.get("rotate_seconds", None),
                    "surface_enabled": metadata.get("surface_enabled",
"surface_temp_c" in data),
                    "dht_enabled": metadata.get("dht_enabled", True),
                })
            required = {"t_perf_ns", "ax_g", "ay_g", "az_g"}
            if data.empty or not required.issubset(data.columns):
                time.sleep(0.3)
                continue
            # 파일 경계에서도 기준 보정/최근 과형을 유지한다. 메모리는 최근 rows개로
제한.
            frames = [frame for frame in (self.recent_data, data) if not
frame.empty]
            data = pd.concat(frames, ignore_index=True)
            data = data.drop_duplicates(subset=["t_perf_ns"], keep="last")
            data = data.sort_values("t_perf_ns").tail(rows).copy()
            self.recent_data = data
            for column in ["t_perf_ns", "t_unix_s", "ax_g", "ay_g", "az_g",
"temp_c", "humidity", "surface_temp_c",
"surface_age_s", "dht_age_s"]:
                if column in data.columns:
                    data[column] = pd.to_numeric(data[column],
errors="coerce")
            data = data.dropna(subset=["t_perf_ns", "ax_g", "ay_g",
"az_g"]).sort_values("t_perf_ns")
            if data.empty:
                time.sleep(0.3)
                continue
            max_t = float(data["t_perf_ns"].iloc[-1])

```



```

        except Exception as exc:
            with self.lock:
                self.state["info"] = f"대기/오류: {exc}"
            time.sleep(0.25)

def get(self) -> dict:
    with self.lock:
        result = dict(self.state)
        for key in [
            "vibration_rms_g",
            "vibration_peak_g",
            "orientation_change_deg",
            "vibration_threshold_g",
            "peak_threshold_g",
            "orientation_threshold_deg",
            "vibration_ratio",
            "peak_ratio",
            "temp_c",
            "surface_temp_c",
            "surface_age_s",
            "dht_age_s",
            "humidity",
            "updated_at",
            "last_alert_at",
            "calibration_progress",
        ]:
            result[key] = finite_or_none(result.get(key))
            elapsed = max(0.0, time.time() - result["updated_at"]) if
result.get("updated_at") else math.inf
            for value_col, age_col, enabled_col, status_col in (
                ("surface_temp_c", "surface_age_s", "surface_enabled",
"surface_status"),
                ("humidity", "dht_age_s", "dht_enabled", "humidity_status"),
            ):
                age = result.get(age_col)
                stale = elapsed > self.env_stale_seconds or (
                    age is not None and age + elapsed > self.env_stale_seconds)
                if not result[enabled_col]:
                    result[value_col], result[status_col] = None, "미사용"
                elif result.get(value_col) is None or stale:
                    result[value_col], result[status_col] = None, "결측 / 센서 확인"
                else:
                    result[status_col] = "정상"
            if self.logger_process is not None:
                exit_code = self.logger_process.poll()
                result["logger_status"] = "수집 중" if exit_code is None else f"로거
종료 (코드 {exit_code})"
            else:
                result["logger_status"] = "기존 기록 연결"
        return result

def local_ip() -> str:
    try:
        result = subprocess.run(["hostname", "-I"], capture_output=True,
text=True, check=False)
        for value in result.stdout.split():
            if ":" not in value and not value.startswith("127."):
                return value
    except OSError:
        pass
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.connect(("10.255.255.255", 1))

```

```

        value = sock.getsockname()[0]
        sock.close()
        return value
    except OSError:
        return "라즈베리파이_IP"

def resolve_logger(path_value: str) -> Path:
    path = Path(path_value)
    if not path.is_absolute() and not path.exists():
        candidate = Path(__file__).resolve().parent / path
        if candidate.exists():
            path = candidate
    if not path.exists():
        raise FileNotFoundError(f"로거 파일을 찾지 못했습니다: {path}")
    return path

def serve(bridge: Path, args: argparse.Namespace) -> None:
    monitor = SecurityMonitor(bridge, args)
    app = Flask(__name__)

    @app.get("/")
    def index():
        return HTML

    @app.get("/api")
    def api():
        return jsonify(monitor.get())

    @app.post("/api/security/arm")
    def arm():
        return jsonify(monitor.arm())

    @app.post("/api/security/disarm")
    def disarm():
        return jsonify(monitor.disarm())

    @app.post("/api/security/reset")
    def reset():
        return jsonify(monitor.reset_alert())

    print(f"보안 대시보드: http://{local_ip()}:{args.port}")
    print("대시보드에서 감시 시작을 누른 뒤 보정 시간 동안 기체를 건드리지 않는다.")
    app.run(host=args.host, port=args.port, threaded=True, use_reloader=False)

def main() -> None:
    args = parse_args()
    process: subprocess.Popen | None = None
    try:
        if args.bridge or args.manifest:
            bridge = Path(args.bridge or args.manifest).resolve()
            if not bridge.exists():
                raise FileNotFoundError(f"기록 파일을 찾지 못했습니다: {bridge}")
        else:
            logger = resolve_logger(args.logger).resolve()
            output = Path(args.output_dir).resolve()
            output.mkdir(parents=True, exist_ok=True)
            # 매 실행마다 고유 폴더를 만든다. 기존 CSV/메타데이터는 삭제하지 않는다.
            run_dir = Path(tempfile.mkdtemp(
                prefix=f"{args.condition}_{time.strftime('%Y%m%dT%H%M%S')}_",
                dir=output))
            bridge = run_dir / "current.json"

```

```

        command = [
            sys.executable, "-u", str(logger),
            "--condition", args.condition, "--run-dir", str(run_dir),
            "--sample-rate", str(args.sample_rate), "--range-g",
str(args.range_g),
            "--rotate-seconds", str(args.rotate_seconds),
            "--flush-seconds", str(args.flush_seconds),
            "--fsync-seconds", str(args.fsync_seconds),
            "--env-stale-seconds", str(args.env_stale_seconds),
        ]
    if args.no_dht:
        command.append("--no-dht")
    if args.no_surface:
        command.append("--no-surface")
    if args.surface_device:
        command.extend(["--surface-device", args.surface_device])
    process = subprocess.Popen(command)
    args.logger_process = process
    deadline = time.monotonic() + 45.0
    while not bridge.exists() and time.monotonic() < deadline:
        if process.poll() is not None:
            raise RuntimeError("로거가 먼저 종료되었습니다.
센서/라이브러리/디스크 오류를 확인하세요.")
        time.sleep(0.2)
    if not bridge.exists():
        raise RuntimeError("로거 시작 시간 초과: current.json이 생성되지
않았습니다.")
    print(f"[저장] 실행 폴더: {run_dir}")
    print(f"[저장] {args.rotate_seconds:g}초마다 새 csv. 기존 파일은
유지됩니다.")
    serve(bridge, args)
    finally:
        if process is not None and process.poll() is None:
            # 수정된 logger.py는 SIGTERM을 받아 큐를 비우고 flush/fsync 후 종료한다.
            process.terminate()
            try:
                process.wait(timeout=25.0)
            except subprocess.TimeoutExpired:
                print("[경고] 로거 종료 지연으로 강제 종료합니다. 마지막 파일을
점검하세요.")
                process.kill()
                process.wait()

if __name__ == "__main__":
    main()

```

logger.py

```

from __future__ import annotations

# 3차 실험: ADXL355 + DS18B20 표면온도 + DHT22 습도.
# 기본: 실행 시작 기준 3600초마다 CSV 교체. 수집 중 계속 기록/flush한다.
# temp_c는 호환성을 위해 DHT22 기온으로 유지하며 표면온도는 surface temp_c이다.
# DS18B20: logger4.py와 같은 1-Wire 설정(GPIO17). 태퍼 코드는 사용하지 않는다.
import argparse
import csv
import json
import math
import multiprocessing as mp
import os
import queue
import re

```

```

import signal
import tempfile
import threading
import time
from pathlib import Path
from typing import Final

ADXL355_ADDR_LOW: Final = 0x1D
ADXL355_ADDR_HIGH: Final = 0x53
REG_DEVID_AD: Final = 0x00
REG_TEMP2: Final = 0x06
REG_XDATA3: Final = 0x08
REG_FILTER: Final = 0x28
REG_RANGE: Final = 0x2C
REG_POWER_CTL: Final = 0x2D
REG_RESET: Final = 0x2F
EXPECTED_ID: Final = (0xAD, 0x1D, 0xED)
RANGE_TO_CODE = {2: 0x01, 4: 0x02, 8: 0x03}
RANGE_TO_SCALE_G_PER_LSB = {
    2: 2.048 / (2**19),
    4: 4.096 / (2**19),
    8: 8.192 / (2**19),
}
RANGE_TO_FULL_SCALE_G = {2: 2.048, 4: 4.096, 8: 8.192}
SAMPLE_RATE_TO_FILTER = {125: 0x05, 250: 0x04, 500: 0x03}
TEMP_BIAS_LSB: Final = 1852.0
TEMP_SLOPE_LSB_PER_C: Final = -9.05

def twos_complement(value: int, bits: int) -> int:
    sign_bit = 1 << (bits - 1)
    return value - (1 << bits) if value & sign_bit else value

def detect_address(bus: SMBus, requested: str) -> int:
    candidates = [ADXL355_ADDR_LOW, ADXL355_ADDR_HIGH] if requested == "auto" else [int(requested, 16)]
    for address in candidates:
        try:
            device_id = tuple(bus.read_i2c_block_data(address, REG_DEVID_AD, 3))
        except OSError:
            continue
        if device_id == EXPECTED_ID:
            return address
    raise RuntimeError("ADXL355를 찾지 못했습니다. i2cdetect -y 1에서 1d 또는 53을 확인해야 합니다.")

def configure_adxl355(bus: SMBus, address: int, range_g: int, sample_rate: int) -> None:
    bus.write_byte_data(address, REG_RESET, 0x52)
    time.sleep(0.1)
    bus.write_byte_data(address, REG_RANGE, 0x80 | RANGE_TO_CODE[range_g])
    bus.write_byte_data(address, REG_FILTER, SAMPLE_RATE_TO_FILTER[sample_rate])
    bus.write_byte_data(address, REG_POWER_CTL, 0x00)
    time.sleep(0.1)

def read_adxl_temperature_c(bus: SMBus, address: int) -> float:
    temp2, temp1 = bus.read_i2c_block_data(address, REG_TEMP2, 2)
    raw = ((temp2 & 0x0F) << 8) | temp1
    return 25.0 + (raw - TEMP_BIAS_LSB) / TEMP_SLOPE_LSB_PER_C

```

```

def read_acceleration_g(bus: SMBus, address: int, scale_g_per_lsb: float) ->
tuple[float, float, float]:
    data = bus.read_i2c_block_data(address, REG_XDATA3, 9)
    values: list[float] = []
    for index in range(0, 9, 3):
        raw20 = (data[index] << 12) | (data[index + 1] << 4) | (data[index + 2] >>
4)
        values.append(twos_complement(raw20, 20) * scale_g_per_lsb)
    return values[0], values[1], values[2]

def wait_until_ns(target_ns: int) -> None:
    while True:
        remaining_ns = target_ns - time.perf_counter_ns()
        if remaining_ns <= 0:
            return
        if remaining_ns > 2_000_000:
            time.sleep((remaining_ns - 1_000_000) / 1e9)
        else:
            time.sleep(0)

def unix_from_perf_ns(perf_ns: int, anchor_perf_ns: int, anchor_unix_s: float) ->
float:
    return anchor_unix_s + (perf_ns - anchor_perf_ns) / 1e9

CSV_COLUMNS = [
    "sample_index", "t_perf_ns", "t_unix_s", "scheduled_perf_ns",
    "lateness_us", "missed_slots_total",
    "temp_c", "humidity", "dht_age_s", "dht_temp_age_s",
    "surface_temp_c", "surface_age_s", "surface_ok", "humidity_ok",
    "adx1_temp_c", "ax_g", "ay_g", "az_g",
]
DS18B20_DEVICES_DIR = Path("/sys/bus/w1/devices")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="SENTRY hourly CSV +
DS18B20/DHT22 logger")
    parser.add_argument("--condition", default="drift-check")
    parser.add_argument("--output-dir", default="data/check")
    parser.add_argument("--run-dir", help="대시보드가 생성한 빈 실행 폴더(내부용)")
    parser.add_argument("--bus", type=int, default=1)
    parser.add_argument("--address", choices=["auto", "0x1d", "0x53"],
default="auto")
    parser.add_argument("--sample-rate", type=int, choices=[125, 250, 500],
default=125)
    parser.add_argument("--range-g", type=int, choices=[2, 4, 8], default=8)
    parser.add_argument("--rotate-seconds", type=float, default=3600.0,
        help="실행 시작 기준 CSV 분할 간격. 기본 1시간")
    parser.add_argument("--flush-seconds", type=float, default=1.0)
    parser.add_argument("--fsync-seconds", type=float, default=10.0,
        help="OS에 파일 동기화 요청 간격. 정전 무손실 보장은 아님")
    parser.add_argument("--dht-interval", type=float, default=2.0)
    parser.add_argument("--surface-interval", type=float, default=2.0)
    parser.add_argument("--adx1-temp-interval", type=float, default=1.0)
    parser.add_argument("--env-stale-seconds", type=float, default=10.0,
        help="마지막 성공한 환경 측정값이 이 시간보다 오래되면 빈 칸")
    parser.add_argument("--surface-device", help="여러 DS18B20 사용 시 28-... 장치
ID")
    parser.add_argument("--no-dht", action="store_true")
    parser.add_argument("--no-surface", action="store_true")

```

```

args = parser.parse_args()
if not re.fullmatch(r"[\w.-]+", args.condition) or args.condition in {".",
".."}:
    parser.error("condition에는 문자, 숫자, _, -, .만 사용할 수 있습니다.")
for name in ("rotate_seconds", "flush_seconds", "fsync_seconds",
            "dht_interval", "surface_interval", "adx1_temp_interval",
"env_stale_seconds"):
    value = getattr(args, name)
    if not math.isfinite(value) or value <= 0:
        parser.error(f"{name}은 유효한 양수여야 합니다.")
if args.dht_interval < 2.0:
    parser.error("dht-interval은 2초 이상이어야 합니다.")
if args.surface_interval < 1.0:
    parser.error("surface-interval은 1초 이상이어야 합니다.")
return args

def create_run_dir(args: argparse.Namespace) -> Path:
    if args.run_dir:
        run = Path(args.run_dir).resolve()
        run.mkdir(parents=True, exist_ok=True)
        if any(run.iterdir()):
            raise FileExistsError(f"실행 폴더가 비어 있지 않습니다. 덮어쓰지 않습니다:
{run}")
        return run
    output = Path(args.output_dir).resolve()
    output.mkdir(parents=True, exist_ok=True)
    return Path(tempfile.mkdtemp(
        prefix=f"{args.condition}_{time.strftime('%Y%m%dT%H%M%S')}_", dir=output))

def sync_directory(path: Path) -> None:
    fd = os.open(str(path), os.O_RDONLY | os.O_DIRECTORY)
    try:
        os.fsync(fd)
    finally:
        os.close(fd)

def atomic_json(path: Path, value: dict) -> None:
    # 동일 파일시스템에서 원자적 교체: 대시보드가 반쪽짜리 JSON을 읽지 않게 한다.
    fd, temporary = tempfile.mkstemp(prefix="." + path.name, dir=path.parent)
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as file:
            json.dump(value, file, ensure_ascii=False, indent=2, allow_nan=False)
            file.flush()
            os.fsync(file.fileno())
        os.replace(temporary, path)
        sync_directory(path.parent)
    finally:
        if os.path.exists(temporary):
            os.unlink(temporary)

def find_ds18b20_device(device_id: str | None = None) -> Path:
    if device_id:
        if not re.fullmatch(r"28-[0-9a-fA-F]+", device_id):
            raise ValueError("surface-device에는 28-... 형식의 장치 ID를 지정하세요.")
        candidates = [DS18B20_DEVICES_DIR / device_id / "w1_slave"]
        candidates = [path for path in candidates if path.exists()]
    else:
        candidates = sorted(DS18B20_DEVICES_DIR.glob("28-*/w1_slave"))
    if not candidates:
        raise RuntimeError(

```

```

        "DS18B20 없음: /boot/firmware/config.txt의 "
        "dtoverlay=w1-gpio,gpiopin=17 설정과 재부팅, 28-* 장치를 확인하세요.")
    if len(candidates) > 1:
        raise RuntimeError("DS18B20이 여러 개입니다. --surface-device 28-... 로
지정하세요.")
    return candidates[0]

def read_ds18b20_c(path: Path) -> float:
    # logger4.py의 CRC 판독 방식. 블로킹 판독은 별도 프로세스에서만 수행.
    try:
        lines = path.read_text().strip().splitlines()
        if len(lines) < 2 or not lines[0].strip().endswith("YES"):
            return math.nan
        value = int(lines[1].partition("t=")[2].strip()) / 1000.0
    except (OSError, ValueError):
        return math.nan
    # logger4.py와 동일하게 파워온 기본값 85°C는 무효로 처리한다.
    return value if -55 <= value <= 125 and value != 85.0 else math.nan

def publish_sensor(output_queue: mp.Queue, item: tuple) -> None:
    try:
        output_queue.put_nowait(item)
    except queue.Full:
        # 최신 측정만 필요. Queue feeder와의 경합 시 다음 측정에서 다시 전달.
        try:
            output_queue.get_nowait()
        except queue.Empty:
            pass
        try:
            output_queue.put_nowait(item)
        except queue.Full:
            pass

def dht_worker(output_queue: mp.Queue, stop_event: mp.Event, interval_s: float) ->
None:
    import adafruit_dht
    import board
    dht = adafruit_dht.DHT22(board.D4)
    try:
        while not stop_event.is_set():
            temp, hum = math.nan, math.nan
            try:
                t, h = dht.temperature, dht.humidity
                if t is not None and math.isfinite(float(t)):
                    temp = float(t)
                if h is not None and math.isfinite(float(h)) and 0 <= float(h) <=
100:
                    hum = float(h)
            except (RuntimeError, OSError):
                pass
            publish_sensor(output_queue, (time.perf_counter_ns(), temp, hum))
            stop_event.wait(interval_s)
    finally:
        dht.exit()

def surface_worker(output_queue: mp.Queue, stop_event: mp.Event,
                    interval_s: float, device_path: str) -> None:
    while not stop_event.is_set():
        value = read_ds18b20_c(Path(device_path))
        publish_sensor(output_queue, (time.perf_counter_ns(), value))

```

```

        stop_event.wait(interval_s)

class SensorCache:
    def __init__(self) -> None:
        self.values: dict[str, tuple[int, float]] = {}

    def update(self, name: str, measured_ns: int, value: float) -> None:
        # 실패한 측정 시도가 아닌, 성공한 값에 대해서만 시각을 갱신한다.
        if math.isfinite(value):
            self.values[name] = (measured_ns, value)

    def read(self, name: str, now_ns: int, max_age_s: float) -> tuple[float, float, bool]:
        if name not in self.values:
            return math.nan, math.nan, False
        measured_ns, value = self.values[name]
        age = max(0.0, (now_ns - measured_ns) / 1e9)
        valid = age <= max_age_s
        return (value if valid else math.nan), age, valid

def format_optional(value: float, digits: int = 3) -> str:
    return "" if not math.isfinite(value) else f"{value:.{digits}f}"

class RotatingCSVWriter:
    """Bounded queue separates CSV flush/fsync/rotation from the sensor loop."""
    def __init__(self, run_dir: Path, metadata: dict, *, queue_size: int = 1000):
        self.run_dir = run_dir
        self.metadata = metadata
        self.manifest = run_dir / "current.json"
        self.pending: queue.Queue = queue.Queue(maxsize=queue_size)
        self.stop_event = threading.Event()
        self.ready = threading.Event()
        self.error: BaseException | None = None
        self.saved_rows = 0
        self.file = None
        self.writer = None
        self.current_path: Path | None = None
        self.previous_path: Path | None = None
        self.part = 0
        self.thread = threading.Thread(target=self._run, name="csv-writer",
daemon=True)
        self.thread.start()
        if not self.ready.wait(30.0):
            self.stop_event.set()
            raise TimeoutError("CSV 저장 스레드 시작 시간 초과")
        self.check()

    def check(self) -> None:
        if self.error is not None:
            raise RuntimeError(f"CSV 저장 실패: {self.error}") from self.error

    def put(self, row: list, sample_ns: int) -> None:
        self.check()
        try:
            self.pending.put_nowait((row, sample_ns))
        except queue.Full as exc:
            raise RuntimeError("디스크 기록이 밀려 저장 큐가 가득 찼습니다. 수집을 중단합니다.") from exc

    def publish(self, status: str) -> None:
        atomic_json(self.manifest, {

```

```

        **self.metadata,
        "current_csv": self.current_path.name if self.current_path else None,
        "previous_csv": self.previous_path.name if self.previous_path else
None,
        "part": self.part, "status": status, "saved_rows": self.saved_rows,
        "manifest_updated_unix_s": time.time(),
    })

def _sync(self) -> None:
    self.file.flush()
    os.fsync(self.file.fileno())

def _open_part(self, part: int) -> None:
    if self.file is not None:
        self._sync()
        self.file.close()
    self.previous_path = self.current_path
    self.part = part
    self.current_path = self.run_dir / f"part_{part:06d}_bridge_log.csv"
    # x 모드: 기존 CSV는 어떤 경우에도 덮어쓰지 않는다.
    self.file = self.current_path.open("x", newline="", encoding="utf-8",
                                       buffering=1024 * 1024)

    self.writer = csv.writer(self.file)
    self.writer.writerow(CSV_COLUMNS)
    self._sync()
    atomic_json(self.current_path.with_name(f"part_{part:06d}_metadata.json"),
{
    **self.metadata, "part": part,
    "segment_start_elapsed_s": (part - 1) *
self.metadata["rotate_seconds"],
    })
    self.publish("running")
    print(f"[저장] 현재 CSV: {self.current_path}", flush=True)

def _run(self) -> None:
    try:
        self._open_part(1)
        self.ready.set()
        next_flush = time.monotonic() + self.metadata["flush_seconds"]
        next_sync = time.monotonic() + self.metadata["fsync_seconds"]
        rotate_ns = int(self.metadata["rotate_seconds"] * 1e9)
        while not self.stop_event.is_set() or not self.pending.empty():
            try:
                row, sample_ns = self.pending.get(timeout=0.1)
            except queue.Empty:
                row = None
            if row is not None:
                elapsed = max(0, sample_ns - self.metadata["anchor_perf_ns"])
                part = 1 + elapsed // rotate_ns
                if part != self.part:
                    self._open_part(part)
                    self.writer.writerow(row)
                    self.saved_rows += 1
                now = time.monotonic()
                if now >= next_flush:
                    self.file.flush()
                    next_flush = now + self.metadata["flush_seconds"]
                if now >= next_sync:
                    self._sync()
                    self.publish("running")
                    next_sync = now + self.metadata["fsync_seconds"]
                self._sync()
                self.publish("stopped")
            except BaseException as exc:

```

```

        self.error = exc
        print(f"[저장 오류] {exc}", flush=True)
    finally:
        self.ready.set()
        if self.file is not None:
            try:
                self.file.close()
            except OSError as exc:
                if self.error is None:
                    self.error = exc

def close(self) -> None:
    self.stop_event.set()
    self.thread.join(timeout=15.0)
    if self.thread.is_alive():
        raise TimeoutError("CSV 마감이 지연되고 있습니다. 디스크 상태를 확인하세요.")
    self.check()

def main() -> None:
    args = parse_args()
    # 하드웨어 모듈은 실제 수집을 시작할 때만 필요하다.
    from smbus2 import SMBus
    run_dir = create_run_dir(args)
    stop = threading.Event()
    for signum in (signal.SIGINT, signal.SIGTERM):
        signal.signal(signum, lambda _signum, _frame: stop.set())
    children = []
    queues = {}
    writer = None
    cache = SensorCache()
    total_samples = missed_slots = read_errors = 0
    clip_counts = {"ax": 0, "ay": 0, "az": 0}
    latest_adxl_temp = math.nan
    last_adxl_temp_ns = 0
    sample_interval_ns = int(1e9 / args.sample_rate)
    scale = RANGE_TO_SCALE_G_PER_LSB[args.range_g]
    clip_limit = 0.995 * RANGE_TO_FULL_SCALE_G[args.range_g]
    surface_path = None
    try:
        if not args.no_surface:
            surface_path = find_ds18b20_device(args.surface_device)
        context = mp.get_context("spawn")
        specs = []
        if not args.no_dht:
            specs.append(("dht", dht_worker, (args.dht_interval,)))
        if surface_path is not None:
            specs.append(("surface", surface_worker, (args.surface_interval,
str(surface_path))))
        for name, target, extra in specs:
            sensor_queue = context.Queue(maxsize=4)
            sensor_stop = context.Event()
            process = context.Process(target=target, args=(sensor_queue,
sensor_stop, *extra),
                                     daemon=True)

            process.start()
            queues[name] = sensor_queue
            children.append((process, sensor_stop, sensor_queue))

    with SMBus(args.bus) as bus:
        address = detect_address(bus, args.address)
        configure_adxl355(bus, address, args.range_g, args.sample_rate)
        anchor_ns, anchor_unix = time.perf_counter_ns(), time.time()
        metadata = {

```

```

        "schema_version": 3, "condition": args.condition,
        "run_id": run_dir.name, "sample_rate_hz": args.sample_rate,
        "range_g": args.range_g, "address_hex": f"0x{address:02X}",
        "filter_register_hex":
f"0x{SAMPLE_RATE_TO_FILTER[args.sample_rate]:02X}",
        "anchor_perf_ns": anchor_ns, "start_unix_s": anchor_unix,
        "rotate_seconds": args.rotate_seconds,
        "flush_seconds": args.flush_seconds, "fsync_seconds":
args.fsync_seconds,
        "env_stale_seconds": args.env_stale_seconds,
        "temperature_source": "DS18B20", "humidity_source": "DHT22",
        "surface_device": str(surface_path) if surface_path else None,
        "surface_enabled": not args.no_surface, "dht_enabled": not
args.no_dht,
        "temp_c_meaning": "DHT22 air temperature (reference only)",
        "dht_age_s_meaning": "age since last successful humidity reading",
    }
    atomic_json(run_dir / "session_metadata.json", metadata)
    writer = RotatingCSVWriter(run_dir, metadata,
queue_size=args.sample_rate * 5)
    print(f"[수집] {args.sample_rate} Hz / ±{args.range_g} g /
{args.rotate_seconds:g}초 분할")
    print(f"[수집] 실행 폴더: {run_dir}\n[수집] Ctrl+C로 안전 종료",
flush=True)
    next_sample_ns = time.perf_counter_ns()
    while not stop.is_set():
        writer.check()
        wait_until_ns(next_sample_ns)
        if stop.is_set():
            break
        now_ns = time.perf_counter_ns()
        if now_ns - next_sample_ns >= sample_interval_ns:
            skipped = (now_ns - next_sample_ns) // sample_interval_ns
            missed_slots += skipped
            next_sample_ns += skipped * sample_interval_ns
            scheduled_ns = next_sample_ns
            for name, sensor_queue in queues.items():
                try:
                    while True:
                        item = sensor_queue.get_nowait()
                        if name == "dht":
                            measured_ns, temp, hum = item
                            cache.update("air", measured_ns, temp)
                            cache.update("humidity", measured_ns, hum)
                        else:
                            measured_ns, surface = item
                            cache.update("surface", measured_ns, surface)
                except queue.Empty:
                    pass
            if now_ns - last_adxl_temp_ns >= int(args.adxl_temp_interval *
1e9):
                try:
                    latest_adxl_temp = read_adxl_temperature_c(bus, address)
                except OSError:
                    latest_adxl_temp = math.nan
                last_adxl_temp_ns = now_ns
                read_start = time.perf_counter_ns()
                try:
                    ax, ay, az = read_acceleration_g(bus, address, scale)
                except OSError:
                    read_errors += 1
                    next_sample_ns += sample_interval_ns
                    continue
                sample_ns = (read_start + time.perf_counter_ns()) // 2

```

```

        for axis, value in (("ax", ax), ("ay", ay), ("az", az)):
            if abs(value) >= clip_limit:
                clip_counts[axis] += 1
            air, air_age, _ = cache.read("air", sample_ns,
args.env_stale_seconds)
            hum, hum_age, hum_ok = cache.read("humidity", sample_ns,
args.env_stale_seconds)
            surface, surface_age, surface_ok = cache.read("surface",
sample_ns, args.env_stale_seconds)
            row = [
                total_samples, sample_ns,
                f"{unix_from_perf_ns(sample_ns, anchor_ns, anchor_unix):.9f}",
                scheduled_ns, f"{{sample_ns - scheduled_ns} / 1000:.3f}",
missed_slots,
                format_optional(air), format_optional(hum),
format_optional(hum_age),
                format_optional(air_age), format_optional(surface),
format_optional(surface_age),
                int(surface_ok), int(hum_ok),
format_optional(latest_adxl_temp),
                f"{ax:.8f}", f"{ay:.8f}", f"{az:.8f}",
            ]
            writer.put(row, sample_ns)
            total_samples += 1
            next_sample_ns += sample_interval_ns

    finally:
        stop.set()
        for process, sensor_stop, sensor_queue in children:
            sensor_stop.set()
        try:
            if writer is not None:
                writer.close()
        finally:
            for process, sensor_stop, sensor_queue in children:
                process.join(timeout=3.0)
                if process.is_alive():
                    process.terminate()
                    process.join(timeout=2.0)
                sensor_queue.close()
            qc = {
                "queued_samples": total_samples,
                "saved_rows": writer.saved_rows if writer else 0,
                "missed_slots": missed_slots, "read_errors": read_errors,
                "clip_counts": clip_counts,
            }
            try:
                atomic_json(run_dir / "qc_summary.json", qc)
            except OSError as exc:
                print(f"[QC 저장 실패] {exc}", flush=True)
            print(f"[종료/QC] {qc}", flush=True)

if __name__ == "__main__":
    main()

```